МЕТОДОЛОГИЧЕСКОЕ РУКОВОДСТВО
ДЛЯ ПРАКТИЧЕСКИХ РАБОТ
ПО ДИСЦИПЛИНЕ КОМПЬЮТЕРНАЯ ГРАФИКА

Авторы: Ротару Лилия, Ошовская Мариана, Карбуне Виорел, Урсу Адриана

Перевод, адаптация и дополнение: Субботкин Всеволод

Введение

Компьютерная графика — это область, занимающаяся созданием, обработкой и отображением изображений с помощью компьютеров. Эта область постоянно развивается и включает в себя генерацию и модификацию двухмерной и трехмерной цифровой графики. Графика использует различные методы и алгоритмы для генерации изображений с помощью компьютера. В настоящее время компьютерная графика имеет широкий спектр применений, от видеоигр и анимации до научных симуляций, автоматизированного проектирования (САПР), визуализации данных и спецэффектов в фильмах. Данный учебный материал основан на графической библиотеке P5.js, которая, в свою очередь, основана на JavaScript.

Эта область включает в себя такие понятия, как:

- Графические примитивы: основные фигуры, такие как точки, линии, круги, многоугольники, используемые для построения более сложных изображений.
- Графические преобразования: геометрические манипуляции, позволяющие перемещать, вращать, масштабировать и искажать графические объекты.
- 3D-моделирование: создание трехмерных объектов с помощью передовых методов, таких как экструзия, скульптурирование и текстурирование.
- Освещение и затенение: моделирование света, теней и эффектов отражения для создания реалистичных изображений.

Новая область графики, которая в последнее время стремительно развивается, — это дополненная реальность (AR) — технология, которая объединяет созданные компьютером виртуальные элементы с реальным миром, обеспечивая интерактивный и улучшенный опыт. AR использует камеры и датчики для наложения 2D или 3D объектов на видеопоток с камеры устройства, создавая иллюзию того, что эти объекты интегрированы в реальную среду. Технология используется в самых разных приложениях.

Практические работы структурированы таким образом, чтобы студенты постепенно знакомились с различными концепциями и методами, используемыми в компьютерной графике, начиная с основ 2D-графики, продолжая 3D-представлениями, способами создания динамических сцен и заканчивая дополненной реальностью и созданием моделей для 3D-принтеров. Каждая работа включает практические задания и

подробное описание необходимых методов и функций, что обеспечивает глубокое понимание каждой концепции, а также критерии оценки выполненных заданий.

По завершении практических работ студенты приобретут необходимые теоретические и практические навыки в области компьютерной графики, применимые в различных технологических и профессиональных сферах.

Требования по безопасности при проведении занятий

Требования безопасности основаны на Законе № 186 от 2008 года, Постановлении Правительства № 95 от 2009 года и Постановлении Правительства № 252 от 2010 года, а именно:

Правила техники безопасности перед началом работы

1. Выполнение лабораторных работ в отсутствие научного руководителя (преподавателя или лаборанта) запрещено.
2. Перед началом работы студенты должны ознакомиться с соответствующим заданием. Студенты, не подготовленные к работе, к выполнению работы не допускаются.
3. Подключение экспериментальной установки к электросети допускается только с разрешения научного руководителя.

Правила техники безопасности в процессе работы

1. Запрещается оставлять устройство под напряжением, менять рабочее место без разрешения руководителя работ.
2. Запрещается менять предохранитель электросети, когда устройство подключено к сети.
3. Запрещается прикасаться к неизолированным частям электрической цепи.
4. Запрещается подвергать части тела облучению, особенно глаза.
5. Запрещается прикасаться рукой к поверхности радиоактивного источника.
6. Радиоактивный источник выдается лаборантом в начале работы и строго необходимо вернуть его после завершения работы.

Правила техники безопасности при выполнении работ

1. По окончании работы отключите экспериментальную установку от электросети, заблокируйте цифровые электронные таймеры.
2. Отсоедините соединительный провод от сети; выньте из розетки электрический кабель с вилкой.
3. Руководитель работ и лаборант должны проверить, отключены ли устройства и выключен ли свет. Кроме того, они должны убедиться в отсутствии запаха горящих материалов, искр и других признаков возгорания.

Действия, предпринимаемые в случае аварий и других критических ситуаций.

1. При нахождении человека под воздействием электрического напряжения необходимо немедленно отключить экспериментальную установку и уведомить руководителя работ. Если пострадавший не потерял сознание, сначала необходимо обеспечить ему покой, затем его следует доставить к врачу или вызвать скорую медицинскую помощь по телефону 112.
2. Если пострадавший потерял сознание и не дышит, после отключения напряжения немедленно провести искусственное дыхание и принять меры по остановке кровотечения. Одновременно необходимо вызвать скорую медицинскую помощь по телефону 112.
3. При травмах, ушибах и других повреждениях необходимо использовать аптечку; следует следовать инструкциям по оказанию первой помощи.
4. При появлении запаха, искр, дыма или других аномальных явлений установку необходимо немедленно отключить.
5. В случае пожара в первую очередь следует использовать углекислотный огнетушитель, а в местах без электропроводки — пенный, песчаный, водяной или асбестовый огнетушитель.

Методика оценки знаний и результатов практической деятельности

По каждой практической работе составляется отчет по образцу, представленному в Приложении 1. Методика оценки лабораторных работ состоит из следующих этапов:

Студент представляет программный код на компьютере, преподаватель оценивает студента на основе:

- ответов на вопросы самооценки, которые задаются после каждой лабораторной работы;
- преподаватель оценивает код в зависимости от соответствия условиям заданий, предложенных для выполнения, в зависимости от функциональности и оптимизации программы, а также критериев оценки, указанных в каждой лабораторной работе.

Студент должен соблюдать двухнедельный срок сдачи лабораторной работы. За каждую неделю задержки преподаватель имеет право снизить оценку студента за соответствующую работу.

Критерии оценивания	**Оценка 9.00 - 10**	**Оценка 7.00 - 8.99**	**Оценка 5.00 - 6.99**	**Оценка ниже 5.00**	**Вес критерия, влияющий на оценку**
Компиляция и запуск программы	Безошибочная компиляция, корректное отображение результатов выполнения программы и соответствие всем требованиям к работе.	Безошибочная компиляция, корректное отображение результатов работы программы и соответствие большинству требований проекта.	Безошибочная компиляция, корректное отображение результатов выполнения программы и частичное соответствие требованиям к работе.	Компиляция программы с ошибками.	0.2
Содержание отчета	Соответствие стандартам отчета содержания и форматирования	Частичное соответствие стандартам содержания и форматирования отчета.	Частичное несоответствие стандартам содержания и форматирования отчета.	Полное несоответствие стандартам содержания и форматирования отчетов.	0.2
В поддержку отчета	Плавное, логически структурированное и исчерпывающее изложение отчета.	Уверенное и беглое изложение отчета.	Делает длительные паузы при представлении отчета.	Он не в состоянии представить доклад.	0.3
Ответы на вопросы по теме лабораторной работы	Оперативно и правильно отвечайте на все заданные вопросы.	Ответьте правильно более чем на 50% вопросов.	Ответьте правильно на 50% вопросов.	Он не может ответить на заданные вопросы.	0.3

I. 2D графика

В компьютерной графике визуальные изображения, композиции, интерфейс и т. д. реализуются как в программировании, через алгоритмы, однако, если в программировании алгоритмы описывают конкретные шаги выполнения, то в компьютерной графике это уже инструкции, описывающие то, что и как надо нарисовать.

Виды графики

- **Растровая** – изображение состоит из точек, называемых пикселями. Их количество определяется шириной и высотой изображения (количество пикселей на ширину × количество пикселей на высоту). Каждый пиксель содержит информацию о цвете обычно по цветовой модели RGB (красный, зеленый, синий) и показатель альфа-канала, иначе говоря, коэффициент непрозрачности.
- **Векторная** – изображение строится на основе геометрической информации, в частности, фигур (примитивы), где они располагаются и как они должны выглядеть (цвет окраски, градиент или фон; цвет и толщина контура; и. т. д.).

Благодаря тем же инструкциям в компьютерной графике изображения одного вида графики возможно преобразовать в другой вид графики и наоборот, в зависимости от сложности самого изображения. Однако, векторная графика без проблем превращается в растровую графику.

Система координат

В качестве системы координат в компьютерной графике используется далеко устоявшееся и всеми известная Декартовая система координат, состоящая из двух осей – оси абсцисс (X, по горизонтали) и оси ординат (Y, по вертикали). Важное примечание заключается в том, что в компьютерной графике точкой начального отсчета является верхний левый угол. Визуальное представление можно увидеть на рисунке 1.1.

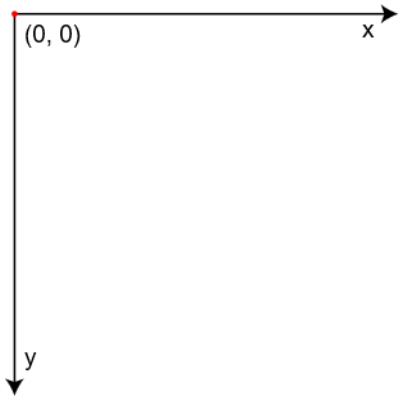


Рисунок 1.1: Система координат, используемая в компьютерной графике.

Библиотека P5.js

На протяжении огромного времени компьютерная графика эволюционировала и инструкции для рендеринга (прорисовки) этой самой графики увеличивались в масштабах и количестве, из-за чего для работы с компьютерной графикой в приложениях переросло в необходимость подключения специализированных библиотек в код тех самых приложений.

На данный момент, лучшим способом для ознакомления с темой компьютерной графики и как она устроена – это библиотека p5 [1] написанная на JavaScript, использующая HTML5 Canvas API [2], что является, в какой-то степени, встроенной библиотекой, и работающая в рамках веб-браузера.

Значительная разница между библиотекой p5 с реализацией кода на JavaScript с прямым нативным обращением к тому самому HTML5 Canvas API – это то, что написание кода с использованием этой библиотеки гораздо проще, если вы новичок в этой теме. Достаточно просто написать код из простых команд – прямо как объяснять, что нужно нарисовать и как.

Пример кода с использованием нативного API:

```
// Берем холст, на котором будем рисовать
var canvas = document.getElementById('canvas'),
// Необходимо получить контекст для рисования
// в нем доступны методы для рисования объектов
    ctx = canvas.getContext('2d');

// Рисуем прямоугольник красного цвета с толстым контуром
ctx.fillStyle = 'red'; // Задаем цвет заливки как красный
ctx.lineWidth = 10; // Делаем прямоугольнику толстый контур
ctx.rect(75, 150, 100, 50); // Рисуем прямоугольник
ctx.fill(); // Красим прямоугольник
ctx.stroke(); // Рисуем контур
```

Посмотреть, как данный код работает можно при помощи веб-песочницы CodePen – https://codepen.io/SVGvsevolod/pen/oNVaPOb, или веб-редактор библиотеки P5.js – https://editor.p5js.org/SVGvsevolod/sketches/3bI576VaM.

Пример кода с использованием API библиотеки p5:

```
function setup() {
  createCanvas(200, 200); // Берем холст
  background(220); // Задаем фон
  fill('red'); // Красим прямоугольник в красный
  strokeWeight(10); // Делаем прямоугольнику толстый контур
  rect(75, 100, 100, 75); // Рисуем прямоугольник
}
```

Посмотреть, как данный код работает можно при помощи веб-редактора библиотеки P5.js – https://editor.p5js.org/SVGvsevolod/sketches/D5CdU-0M0.

Как писать код и веб-редактор p5

Помимо самой библиотеки, для удобства еще предоставляется редактор, в котором можно написать код и сразу посмотреть результат и сам доступен для использования прямо в веб браузере по адресу https://editor.p5js.org/.

В данном редакторе все коды для реализации изображения называются «скетчами». Каждый новый скетч создается со следующим шаблоном кода:

```
function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
}
```

По умолчанию, на вооружение, предоставляется объявление двух следующих функциий:

- **setup()** – данная функция предназначена для первоначальных установок и подготовки вашего «скетча». В рамках первой практической работы ее будет вполне достаточно в коде «скетча».
- **draw()** – данная функция предназначена для покадровой перерисовки изображения, иными словами – анимации. Непрерывный цикличный вызов этой функции можно прервать через функцию **noLoop()**.

Важное примечание: в JavaScript, если в коде две и более функции с одним и тем же именем, то последнее объявление этой функции перезаписывает все предыдущие [7].

Также в шаблоне используются следующие функции библиотеки:

- **createCanvas(ширина, высота, *[рендерер]*)** – создание холста, на котором будут рисоваться дальнейшие объекты. **Рендерер** – это система прорисовки, используемые в работе с холстом. По умолчанию – **'P2D'**, для работы с двухмерной графикой. Можно указать **'WEBGL'** для работы с трехмерной графикой.
- **background(цвет, *[непрозрачность]*)** – задает фон холста.

1.1. Рисование примитивов

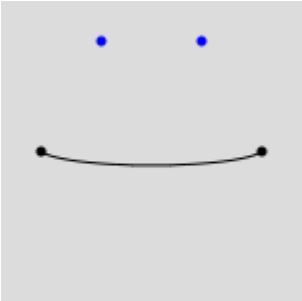
Графические примитивы – это геометрические фигуры, которые лежат в основе компьютерной графике. Библиотека P5.js предоставляет соответствующие функции для их рисования на холсте.

- **point(x, y, *[z]*)** – рисует обычную точку.
- **line(x1, y1, x2, y2)** – рисует линию от начальной точки (x1, y1) до конечной точки (x2, y2).

- **curve(x1, y1, x2, y2, x3, y3, x4, y4)** – кривая линия от (x2, y2) до (x3, y3), где контрольные точки (x1, y1) и (x4, y4) формируют кривизну линии. Пример использования функции можно увидеть в таблице 1.1.

Таблица 1.1: Пример использования функции curve()

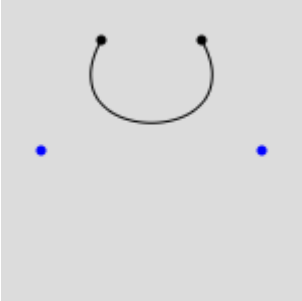
Код	Результат

`function setup() {`
`  createCanvas(150, 150);`
`  background(220);`
`  noFill();`
`  curve(50, 20, 20, 75, 130, 75, 100, 20);`
`  strokeWeight(5);`
`  point(20, 75);`
`  point(130, 75);`
`  stroke('blue');`
`  point(50, 20);`
`  point(100, 20);`
`}`
	

- **bezier(x1, y1, x2, y2, x3, y3, x4, y4)** – кривая Безье. В отличии от функции curve(), точки (x2, y2) и (x3, y3) являются контрольными. Пример использования функции можно увидеть в таблице 1.2.

Таблица 1.2: Пример использования функции bezier()

Код	Результат

`function setup() {`
`  createCanvas(150, 150);`
`  background(220);`
`  noFill();`
`  bezier(50, 20, 20, 75, 130, 75, 100, 20);`
`  strokeWeight(5);`
`  point(50, 20);`
`  point(100, 20);`
`  stroke('blue');`
`  point(20, 75);`
`  point(130, 75);`
`}`
	

- **circle(x, y, диаметр)** – круг.
- **ellipse(x, y, ширина, *[высота]*)** – эллипс.

Примеры использования функций ellipse() и circle() можно увидеть в таблице 1.3.

Таблица 1.3: Примеры использования функций ellipse() и circle()

Код	Результат

`function setup() {`
`  createCanvas(100, 100);`
`  background(220);`
`  ellipse(50, 50, 80, 40);`
`}`	

`function setup() {`
`  createCanvas(100, 100);`
`  background(220);`
`  ellipse(50, 50, 80/*, 80*/);`
`}`	

`function setup() {`
`  createCanvas(100, 100);`
`  background(220);`
`  circle(50, 50, 80);`
`}`	

- **arc(x, y, ширина, высота, начальный угол, конечный угол, *[режим]*)** – дуга.

Начальные и **конечные углы** задаются, по умолчанию, в **радианах**, либо можно в градусах, поменяв режим градусов через функцию **angleMode(DEGREES)**. У библиотеки P5.js есть константы для значений градусов, которые представлены на рисунке 1.2.

Из ***[режимов]*** доступно – **'CHORD'** (пояс), **'PIE'** (пирог) и **'OPEN'** (открытая). По умолчанию – 'OPEN'.

Примеры использования функции можно увидеть в таблице 1.4.

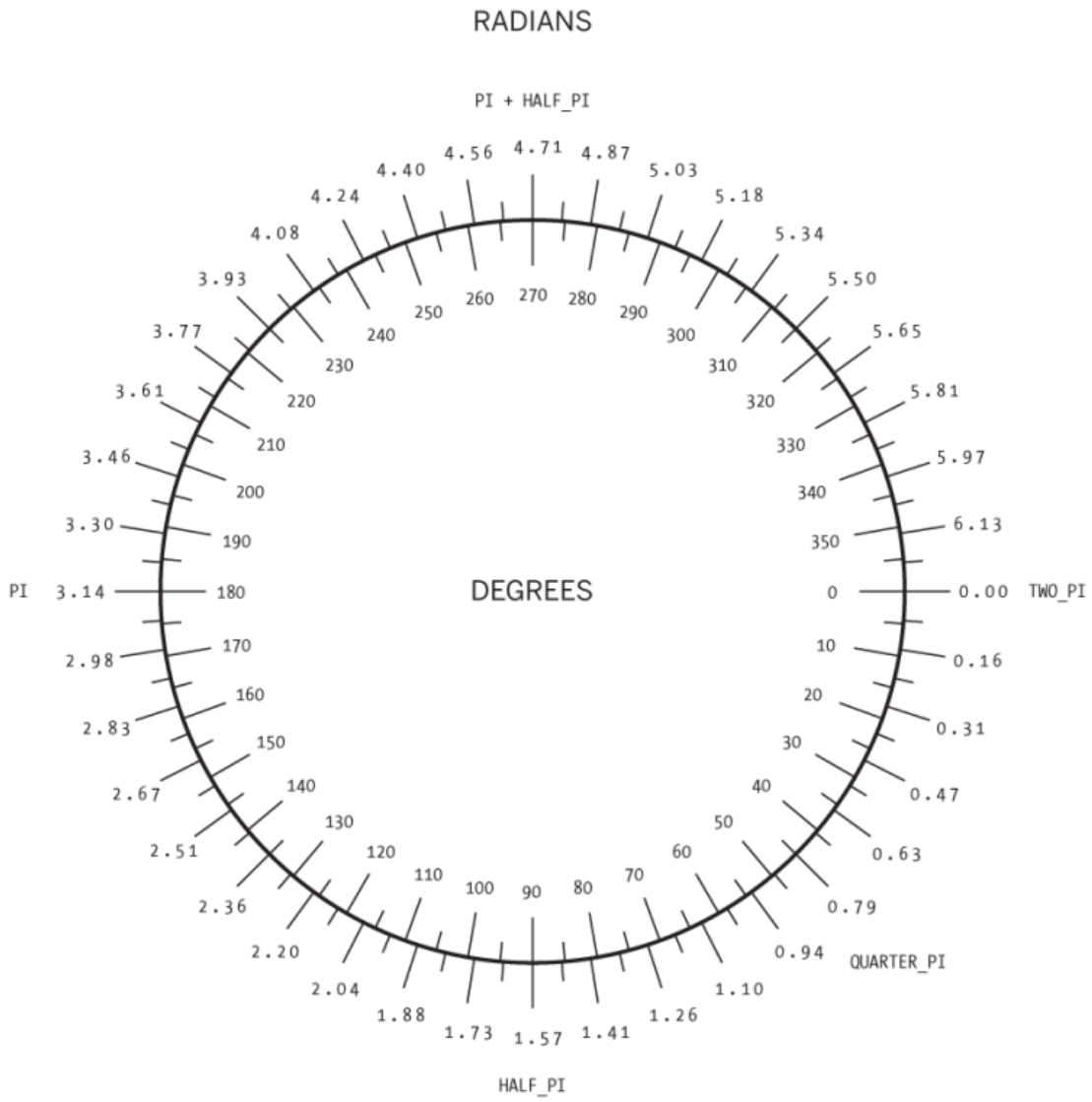
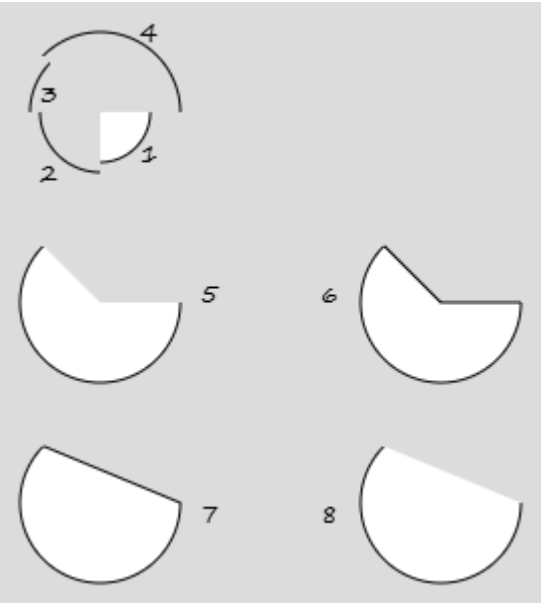


Рисунок 1.2: Взаимосвязь между константами, радианами и градусами.

Таблица 1.4: Примеры использования функции arc()

Код	Результат

`function setup() {`
`  createCanvas(270, 300);`
`  background(220);`
`  arc(50, 55, 50, 50, 0, HALF_PI); //1`
`  push();`
`  noFill();`
`  arc(50, 55, 60, 60, HALF_PI, PI); //2`
`  arc(50, 55, 70, 70, PI, PI +`
`QUARTER_PI); //3`
`  arc(50, 55, 80, 80, PI + QUARTER_PI,`
`TWO_PI); //4`
`  pop();`
`  arc(50, 150, 80, 80, 0, PI +`
`QUARTER_PI); //5`
`  arc(220, 150, 80, 80, 0, PI +`
`QUARTER_PI, PIE); //6`
`  arc(50, 250, 80, 80, 0, PI +`
`QUARTER_PI, CHORD); //7`
`  arc(220, 250, 80, 80, 0, PI +`
`QUARTER_PI, OPEN); //8`
`  text(1, 70, 80);`
`  text(2, 20, 90);`
`  text(3, 20, 50);`
`  text(4, 70, 20);`
`  text(5, 100, 150);`
`  text(6, 160, 150);`
`  text(7, 100, 260);`
`  text(8, 160, 260);`
`}`
	

У примитивов circle(), ellipse() и arc() в библиотеке P5js по умолчанию точка опоры в центре примитива. Поменять это можно при помощи функции **ellipseMode(режим)**. Из доступных **режимов** – **RADIUS** (центр окружности), **CENTER** (тоже центр окружности, но относительно предыдущей окружности), **CORNER** (левый-верхний край примитива) и **CORNERS** (тоже левый-верхний край, но относительно предыдущего примитива). Увидеть разницу между режимами можно увидеть в таблице 1.5.

Таблица 1.5: Примеры разницы режимов функции ellipseMode()

Код	Результат

`function setup() {`
`  createCanvas(100, 100);`
`  background(220);`
`  // Белый эллипс`
`  ellipseMode(RADIUS);`
`  fill(255);`
`  ellipse(50, 50, 30, 30);`
`  // Серый эллипс`
`  ellipseMode(CENTER);`
`  fill(100);`
`  ellipse(50, 50, 30, 30);`
`}`	

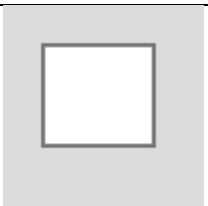
`function setup() {`
`  createCanvas(100, 100);`
`  background(220);`
`  // Белый эллипс`
`  ellipseMode(CORNER);`
`  fill(255);`
`  ellipse(25, 25, 50, 50);`
`  // Серый эллипс`
`  ellipseMode(CORNERS);`
`  fill(100);`
`  ellipse(25, 25, 50, 50);`
`}`	

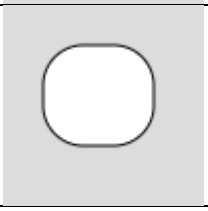
- **triangle(x1, y1, x2, y2, x3, y3)** – треугольник.
- **rect(x, y, ширина, *[высота], [скругление каждого угла по часовой стрелке]*)** – прямоугольник. Для прямоугольников и квадратов есть функция **rectMode()**, которая работает аналогично функции ellipseMode().

Примеры использования функции можно увидеть в таблице 1.6.

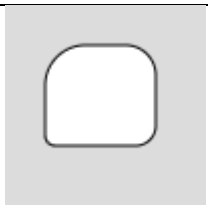
Таблица 1.6: Примеры использования функции rect()

Код	Результат

`function setup() {`
`  createCanvas(100, 100);`
`  background(220);`
`  rect(20, 20, 55, 50);`
`}`
	

`function setup() {`
`  createCanvas(100, 100);`
`  background(220);`
`  rect(20, 20, 55, 50, 20/*, 20, 20, 20*/);`
`}`
	

```
function setup() {
  createCanvas(100, 100);
  background(220);
  rect(20, 20, 55, 50, 20, 15, 10, 5);
}
```



- **square(x, y, размер)** – квадрат.
- **quad(x1, y1, x2, y2, x3, y3, x4, y4)** – четырехугольник. В отличие от rect(), точки фигуры можно ставить произвольно на холсте, тем самым получая нестандартную форму фигуры.
- **text('текст', x, y, *[ширина коробки для текста]*, *[высота коробки]*)** – текст.

Произвольные фигуры

Для рисования произвольных фигур используется связка из функций **beginShape(тип фигуры)**, **vertex(x, y, *[z]*)** и **endShape(*[CLOSE]*)**. beginShape() обозначает начало построения произвольной фигуры, vertex() является одной из гранью произвольной фигуры и endShape() замыкает произвольную фигуру.

Типы фигур для функции beginShape() следующие – **POINTS** (грани рисуются по точкам), **LINES** (рисуются чередующие грани), **TRIANGLE** (чередующие треугольные грани), **TRIANGLE_FAN** (соединенные треугольники, имеющие общую вершину), **TRIANGLE_STRIP** (соединение треугольники как полосы), **QUADS** (чередующие четырехугольники), **QUAD_STRIP** (четырехугольники со смежными ребрами).

Помимо vertex(), есть еще и **curveVertex(x, y, *[z]*)**, **bezierVertex(x2, y2, x3, y3, x4, y4)** и **quadraticVertex(cx, cy, x3, y3)** для кривых граней. curveVertex() работает из связки четырех вызовов этой функции – первый и четвертый – это контрольные точки. Принцип такой же как у функции curve().

Пример использования функции curveVertex():

```
beginShape();
// Первая контрольная точка.
curveVertex(84, 91);
// Сама грань.
curveVertex(68, 19);
curveVertex(21, 17);
// Вторая контрольная точка.
curveVertex(32, 91);
endShape();
```

Функции bezierVertex() и quadraticVertex() требуют использование функции vertex() для остальных точек граней.

В функцию endShape() можно добавить аргумент **CLOSE** для того, чтобы до рисовалась грань замыкающая фигуру.

1.2. Атрибуты

Атрибуты – это параметры, отвечающие за то, как примитивы должны выглядеть. В частности, это заполнение фигуры, контур фигуры (цвет, толщина, стиль). Также такие значения как координаты, ширина, высота и прочие значения размера тоже являются атрибутами. Однако в парадигме библиотеки P5.js, координаты и размеры являются частью функций примитивов, для заполнения и контура есть отдельные, но универсальные функции:

- **fill(цвет, *[непрозрачность]*)** – заполнение (закрашивание) фигуры.
- **noFill()** – не закрашивать.
- **stroke(цвет, *[непрозрачность]*)** – **цвет(!!)** контура.
- **noStroke()** – убрать контур.
- **strokeWeight(толщина)** – толщина контура.
- **strokeCap(тип кончика контура)** – стиль кончиков контура. **Типы кончиков контура** следующие – **ROUND** (скругленные, по умолчанию), **SQUARE** (квадратные) и **PROJECT** (тоже квадратные, на такой же ширины как ROUND).
- **strokeJoin(тип ребра контура)** – стиль ребра контура. Типы ребер следующие – **MITER**, **BEVEL** и **ROUND**. Разницу можно увидеть на рисунке 1.3.

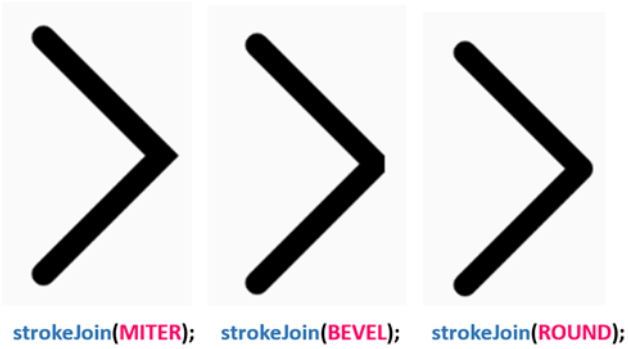


Рисунок 1.3: Разница между типами ребер контура.

- **textAlign(горизонтальное положение, *[вертикальное]*)** – расположение текста. Горизонтальное расположение – это слева (**LEFT**), справа (**RIGHT**) и по центру (**CENTER**). Вертикальное – это сверху (**TOP**), снизу (**BOTTOM**), по центру (**CENTER**) и по основной линии (**BASELINE**).
- **textSize(размер)** – размер текста.
- **textFont('название шрифта', *[размер]*)** – шрифт текста. Можно также указать ***[размер]***.

1.3. Работа с цветом

В компьютерной графике для задания цвета используется цветовая модель с числовыми значениями определяющие тот самый цвет. Распространенной цветовой моделью является RGB – красный, зеленый и синий. Цвет формируется из числовых коэффициентов трех цветов этой цветовой модели в диапазоне от 0 до 255. Дополнительно подключается альфа-канал для задания непрозрачности, с тем же диапазоном от 0 до 255.

Первым способом задания цвета через функции библиотеки p5 работающие со цветом (background(), fill() и stroke()) – указывание этих самых трех коэффициентов как три аргумента функции:

```
background(48, 138, 217);
fill(97, 2, 7);
stroke(206, 219, 15);
```

Дополнительно можно добавить четвертый аргумент для задания непрозрачности:

```
fill(97, 2, 7, 180);
stroke(206, 219, 15, 90);
```

Если в цветовой модели RGB все три коэффициента будут одинакового значения, то цвет будет варьироваться между черным, белым и остальными оттенками серого. В случае функций библиотеки p5, можно указать этот коэффициент как единственный аргумент функции:

```
background(255)
fill(150);
stroke(0);
```

Также помимо RGB есть и другие цветовые модели (CMYK, HSB и HSL, p5 поддерживает только HSB и HSL), а также функции библиотеки p5 принимают строковое значение цвета как аргумент функции:

```
background('rgb(48, 138, 217)');
fill('hsl(357, 96%, 19%)');
stroke('hsb(64, 93%, 86%)');
```

Еще коэффициенты по цветовой модели RGB можно представить в шестнадцатеричном формате:

```
background('#308ad9'); background('#ffffff'); background('#fff');
fill('#610207'); fill('#969696');
stroke('#cedb0f'); stroke('#000000'); stroke('#000');
```

В шестнадцатеричном формате тоже можно еще и указать коэффициент непрозрачности:

```
fill('#61020788');
stroke('#cedb0f33');
```

Еще библиотекой P5.js принимаются распространенные названия цветов:

```
background('grey');
fill('green');
stroke('red');
```

Для упрощения подбора нужного цвета, в поисковике Google можно дать поисковый запрос на словосочетание «color picker» и в результатах будет предоставлен инструмент для подбора цвета и поля, в которых будут написаны коэффициента для копирования – https://www.google.com/search?q=color+picker. Значения этих полей можно скопировать и использовать в аргументах цветовых функций, в частности значение поля RGB – это числовые значения аргументов цветовых функций. На рисунке 1.4. представлен инструмент выбора цвета доступный в поисковике Google.

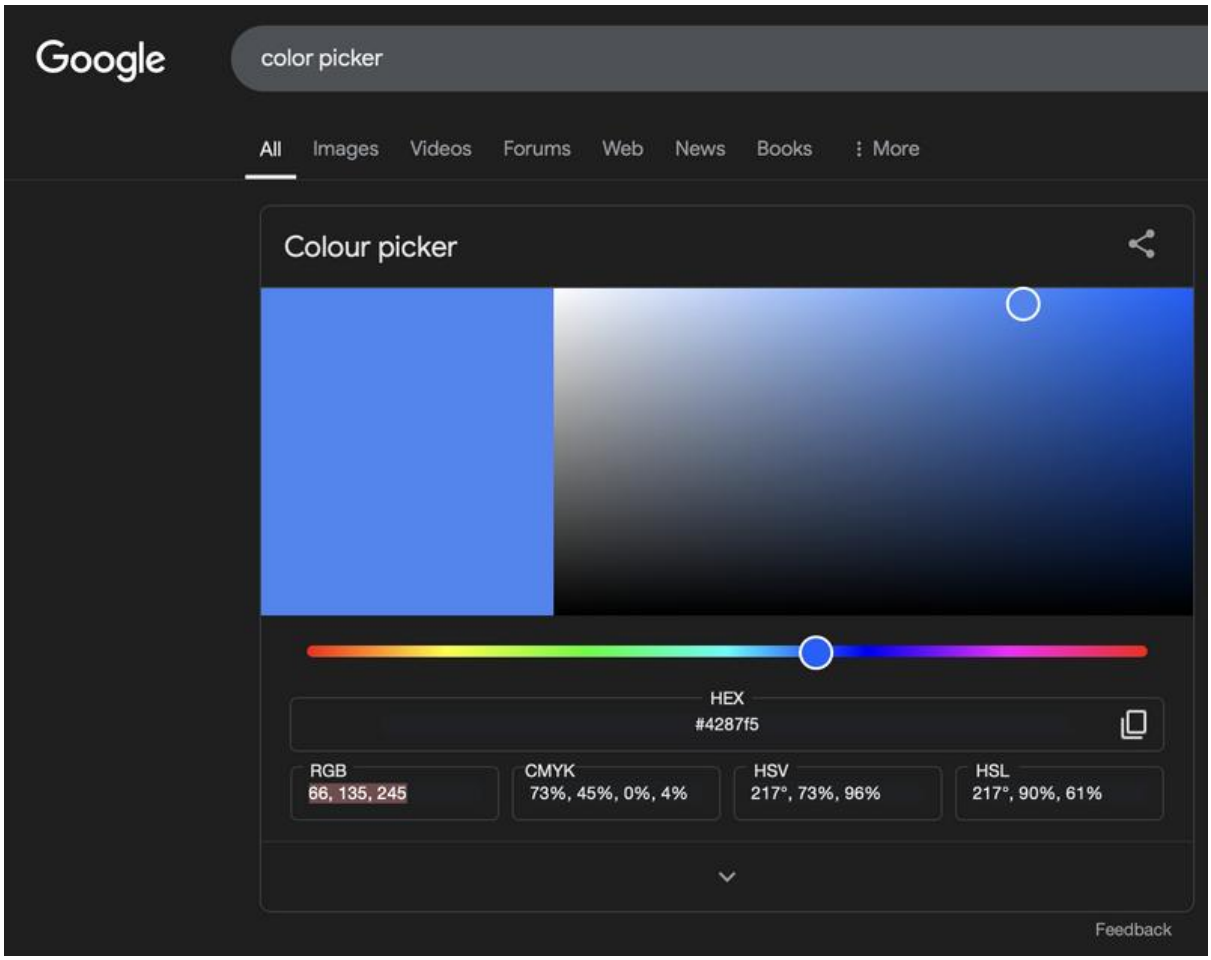


Рисунок 1.4: Инструмент выбора цвета доступный в поисковике Google.

1.4. Переменные

В языках программирования есть переменные – условные коробочки для хранения информации. В компьютерной графике использование переменных тоже не является исключением.

В JavaScript для задания переменной есть ключевые слова **var** или **let**. Разница между ними заключается в принципе работы.

```
var x; // только объявление переменной, значение будет undefined
x = 12; // присваивание значения (инициализация), однако если переменная не была
объявлена, то также она будет объявлена, а потом инициализирована
var x = 12; // объявление и инициализация сразу

let x = 12; // объявление и инициализация через ключевое слово let
```

В первую очередь играет ключевую роль контекст (scope) – переменная может быть глобальной если объявлена вне каких-либо функций или других блоков (внутри фигурных скобок {}) и станет частью объекта window (в случае если код выполняется в браузере), а переменные, объявленные внутри функций или блоков являются локальными и будут уничтожены, если к ним нет привязки из вне.

Значительная разница let от var – это защитные механизмы. var переменные, перезаписываемые из внутренних блоков, а let переменные привязываются к блокам и контекстам. let переменные стоит сразу инициализировать в отличие от var, т. к. можно получить мертвую зону. let переменные нельзя перезаписывать внутри одного контекста, однако можно перезаписать внешнюю из внутреннего блока.

Еще можно задавать константы при помощи ключевого слова **const** вместо let или var.

Еще библиотека P5.js задает глобальные переменные, которые можно использовать в коде для удобства:

- **width, height** – ширина и высота холста.
- **mouseX, mouseY** – текущая позиция курсора мыши.
- **pmouseX, pmouseY** – предыдущая позиция курсора мыши.
- **movedX, movedY** – разница смешения курсора мыши.
- **mouseIsPressed** – логическая переменная, сообщающая о том, что нажата кнопка мыши или нет.
- **keyIsPressed** – логическая переменная, сообщающая о том, что нажата клавиша на клавиатуре или нет.

1.5. Помощь с координатами

Координаты – неотъемлемая составляющая компьютерной графики. Два числовых значения определяющих расположение визуальных объектов на полотне. Вполне удобно будет если знать точные значения, нежели гадать путем инкрементации и декрементации значения, пока объект не будет находиться в подходящем месте. Файл **coords.js** – это помощник с координатами. Он показывает координаты в соответствии с расположением курсора мыши на полотне.

1. Скачайте файл **coords.js** по этой ссылке - https://gist.githubusercontent.com/SVGvsevolod/8bc29908469596b1132579480e7e9e95/raw/8c0ffd37a45c08326db37732e59ac8f4830dd70e/coords.js. Правая кнопка мыши после того как открыли ссылку -> Сохранить как -> Сохраняете в предпочитаемом месте.
2. В редакторе p5 нажимаем на > чтобы открыть список файлов скетча. Показано на рисунке 1.5.

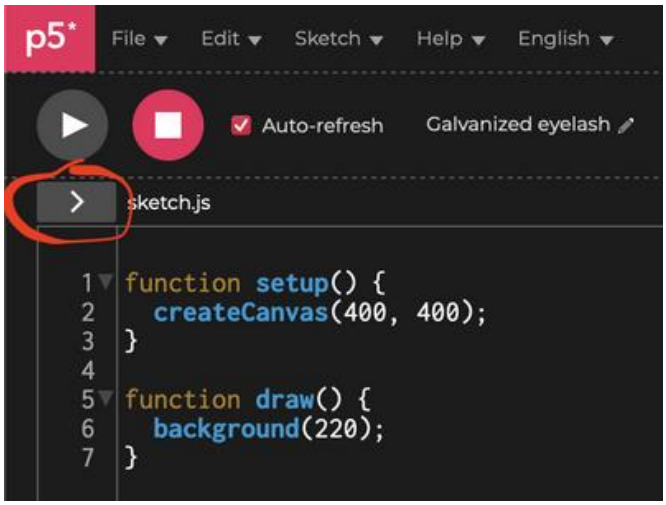


Рисунок 1.5: Кнопка для отображения файлов «скетча»

3. Если не авторизована учетная запись в редакторе, то открываем скаченный js файл в блокноте, выделяем и копируем весь код, жмем + и пункт создать файл (**Create file**), в появившемся окне, в поле пишем **coords.js**, создаем файл (**Add File**) и вставляем код. Либо если авторизована учетная запись, выбираем пункт загрузки файла (**Upload file**), в появившемся окне перетаскиваем в него скачанный файл и закрываем это окно. Показано на рисунке 1.6.

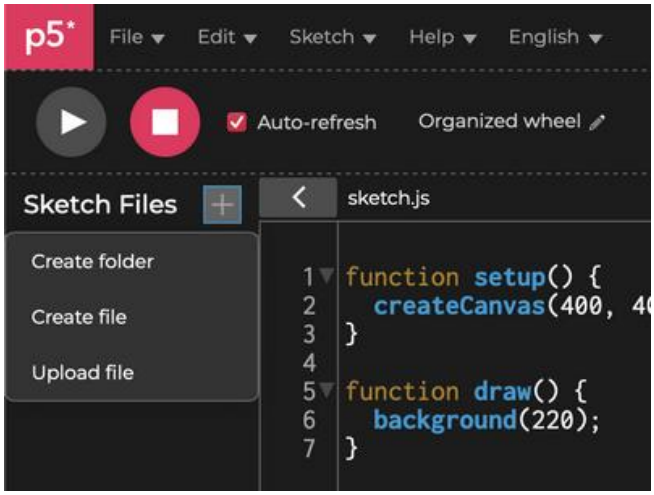


Рисунок 1.6: Добавление нового файла в «скетч»

4. Переключаемся на **index.html** и дописываем следующую строчку, чтобы помощник начал работать:

```
<script src="coords.js"></script>
```

- Функция coords(x, y) отображает прямоугольник с координатами и рекомендуется ее использовать внутри функции draw() или loop()
- Левый клик мыши по любому месту на полотне выведет координаты в консоль для копирования
- Правый клик мыши перемещает прямоугольник с координатами в место, где был курсор в момент клика
- По умолчанию, помощник отключает контекстное меню у полотна. Включить обратно можно данной строкой кода:

```
coords.disableContextMenu = false;
```

1.6. Форматы хранения графического изображения

Любое графическое изображение сохраняется в файле. Способ хранения графических данных в файле определяет графический формат файла.

Формат — структура файла, определяющая способ хранения и отображения данных на экране или при печати.

Формат файла указывается в его имени частью, отделенной точкой (эта часть называется расширением имени файла).

Для графических файлов используется сжатие, поскольку они содержат достаточно большой объем информации, и каждый формат имеет свой собственный алгоритм сжатия данных, содержащихся в файле.

Графические форматы классифицируются следующим образом:

- тип хранимых данных (растровые, векторные и смешанные фигуры);
- в зависимости от допустимого объема данных;

- параметры изображения;
- способ хранения цветовой палитры;
- метод сжатия данных;
- методы организации файла (текстовый, бинарный).

Из множества форматов нет идеального, который бы удовлетворял всем возможным требованиям. Выбор того или иного формата для сохранения изображений зависит от целей и задач работы с изображением. Если требуется качественная точность цветопередачи, предпочтительнее использовать растровые форматы. Для логотипов, диаграмм, элементов дизайна рекомендуются векторные форматы хранения. Формат файла влияет на объем памяти, занимаемый файлом. Графические редакторы позволяют пользователю самостоятельно выбирать формат сохранения изображения. Существуют универсальные графические форматы файлов, поддерживающие одновременно как векторные, так и растровые изображения.

Практическая работа №1

Тема работы: Работа с 2D графикой

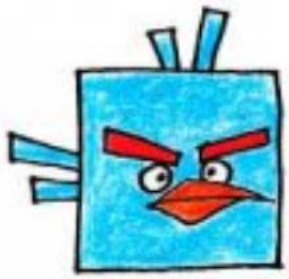
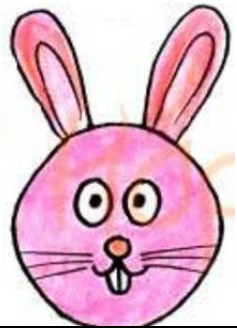
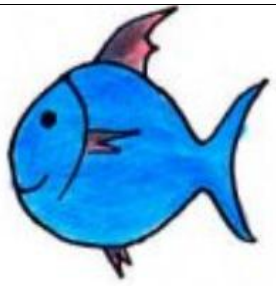
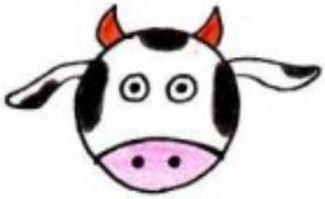
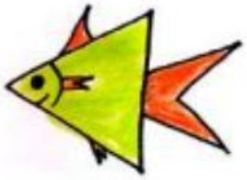
Основные задачи

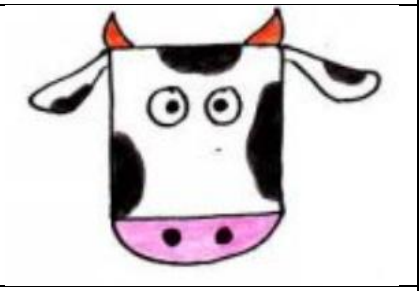
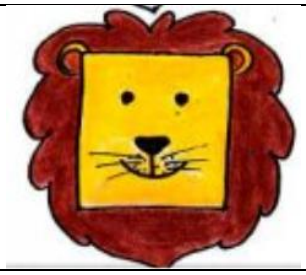
Подготовка к работе: рекомендуется создать учетную запись в редакторе библиотеки P5.js – https://editor.p5js.org/. В редакторе код сохраняются в «скетчах» и для удобства, можно предоставить Ваш код через ссылку на «скетч», чтобы можно было легко и быстро запустить Ваш код и проверить Вашу работу.

Реализация кода: необходимо реализовать код, который в качестве результата нарисует композицию (2D сцену). Композиция должна состоять из, как минимум, одного объекта (или субъекта) и фона. Необходимо задействовать по одному из различных примитивов – point(), line(), curve(), bezier(), ellipse(), circle(), arc(), triangle(), rect(), square() и quad(); максимум 6 различных. Примитивы должны иметь различные атрибуты, работа должна быть подписана (фамилия, имя, группа) в правом нижнем углу экрана при помощи text(). Варианты для реализации предоставлены в таблице 1.8., а также пошаговая инструкция доступна по следующей ссылке – https://ja-rastu.ru/raskraski/uroki/1079-poshagovoe-risovanie-zhivotnyh-iz-geometricheskih-figur.html.

Таблица 1.8: Варианты для реализации.

Вариант	Цель для реализации	Вариант	Цель для реализации
1		11	

Вариант	Цель для реализации	Вариант	Цель для реализации
2		12	
3		13	
4		14	
5		15	
6		16	
7		17	

Вариант	Цель для реализации	Вариант	Цель для реализации
8		18	
9		19	
10		20	

Пример реализации:

```
function setup() {
  createCanvas(300, 225);
  background(200);
  fill(0);
  circle(150,120,150);
  fill(255);
  circle(120,100,40);
  circle(180,100,40);
  fill(0);
  circle(120,100,10);
  circle(180,100,10);
  circle(100,50,60);
  circle(200,50,60);
  fill(255);
  triangle(130,120,160,120,145,135);
  fill(255);
  ellipse(145,160,60,40);
  noFill();
  beginShape();
  vertex(130,165);
  bezierVertex(150,150,150,150,145,150);
  bezierVertex(150,150,160,170,160,160);
  endShape();
}
```

Результат выполнения кода можно увидеть на рисунке 1.7., а также доступен для просмотре в редакторе библиотеки P5.js – https://editor.p5js.org/SVGvsevolod/sketches/4dgNbIEZr.

Рисунок 1.7: Результат выполнения кода

Критерии оценивания:

- **Правильность написания кода (10%)** – если есть в коде синтаксические или функциональные ошибки.
- **Использование графических примитивов (10%)** – насколько разнообразен и креативен результат применения графический примитивов в соответствии с поставленной задачей.
- **Креативность и комплексность композиции (10%)** – насколько хорошо взаимосвязаны между собой примитивы и насколько хорошо продумана композиция.
- **Соответствие с поставленной задачей (10%)** – с задачей лабораторной работы.
- **Оптимизация кода (10%)** – использование минимального необходимого набора функций, исключение бесполезных алгоритмов и отсутствие перегруженности ресурсов исполняемой среды.
- **Интерактивность** *(не обязательно)* – элементы в результате, с которыми можно взаимодействовать как-либо.
- **Визуальная эстетика (10%)** – насколько приятно для глаза результат.
- **Соблюдение срока (10%)** – каждые 4 недели шансы на высокую оценки снижаются.
- **Оценка знаний (20%)** – насколько хорошо разбираетесь в технических материалах, к примеру, документация по командам или основы компьютерной графики.

Градация по оценкам: 5 – 1–5 пункты обязательны, 6 – 8 – по одному баллу за критерий, 9 – 2 балла.

Вопросы для оценки знаний:

- Перечислите простые графические примитивы.
- Как можно изменить атрибуты отображения графических примитивов?
- Как можно написать текст в графическом режиме?
- Назовите стандартные форматы изображений.
- Перечислите простые графические примитивы, обычно используемые в графических библиотеках.
- Какие графические примитивы можно использовать для создания базовых фигур в 2D-графической сцене?
- Какие функции используются для изменения цвета и/или толщины линии или раскраски графических фигур?
- Как можно установить прозрачность или стиль отображения графических примитивов?
- Объясните, как использовать функции контура и заливки в p5.js.
- Какие функции используются для отображения текста в графической сцене?
- Как можно изменить размер, шрифт и цвет отображаемого текста в p5.js?
- Объясните, как можно расположить текст в графической сцене.
- Перечислите некоторые распространенные форматы растровых изображений.
- В чем разница между JPEG, PNG и GIF?
- Какой формат изображения лучше всего подходит для прозрачности и почему?

II. Векторная графика

Векторная графика — это тип компьютерной графики, использующий математическое описание геометрических фигур (линий, кругов, прямоугольников, многоугольников и т. д.), в отличие от растровой графики, основанной на пикселях. Она используется для создания изображений, которые можно масштабировать или трансформировать без потери качества, поскольку не основана на фиксированной сетке пикселей.

2.1. Характерные черты векторной графики

- **Независимость от разрешения:** векторные изображения можно изменять в размере, сохраняя при этом их четкость и детализацию. При увеличении или уменьшении математические уравнения, лежащие в основе векторной графики, будут пересчитаны, поэтому изображения «перерисовываются», и потери качества не произойдет, что делает их идеальными для печати и использования на различных устройствах с разным разрешением.
- **Геометрические компоненты:** векторная графика строится из геометрических объектов, определяемых математическими формулами. Например, окружность определяется своим радиусом и координатами центра, линия — начальной и конечной точками, а многоугольник — координатами своих вершин.
- **Небольшой размер файлов:** поскольку векторная графика описывается геометрическими данными и/или математическими формулами, она занимает меньше места в физических файлах, чем растровые изображения (которые состоят из сетки пикселей). Это делает их эффективными для хранения и обмена.
- **Возможность редактирования:** векторные объекты можно редактировать по отдельности, не затрагивая другие части изображения. Например, круг можно изменить, не влияя на другие элементы графической сцены, что делает работу с векторной графикой более гибкой, чем с растровой.
- **Области применения:** векторная графика используется для создания логотипов, иконок, иллюстраций, в типографике и технических чертежах. Другая область — графический дизайн — создание рекламных материалов, плакатов, листовок и других графических продуктов. Векторная графика используется для создания анимации, поскольку позволяет легко перемещать и масштабировать графические элементы, к примеру, анимированные стикеры в социальных сетях и/или мессенджерах.

Достоинства векторной графики:

- **Масштабируемость:** отсутствие потери качества при изменении размера.
- **Редактируемость:** легко изменять и настраивать каждый элемент.
- **Небольшой размер файла:** более эффективно для хранения и обмена, в частности, для веб-сайтов и/или веб-приложений.
- **Стабильное качество:** видимая пикселизация (то бишь те самые пиксели становятся визуально очевидными) не проявляется явно независимо от их размера, в отличие от растровых изображений.

Недостатки:

- **Сложность:** Создание изображений с высокой детализацией может потребовать больше времени на работу.
- **Ограничения в плане детализации:** Векторная графика лучше подходит для четких, точных форм, но может быть сложнее в использовании для изображений с высокой детализацией или сложными эффектами освещения и текстуры.

2.2. Формат SVG

Формат **SVG** (Scalable Vector Graphics) используется для описания двухмерных изображений на основе синтаксиса **XML**. SVG — это стандарт W3C (World Wide Web Consortium), разработка которого началась в 1999 году. Будучи текстовой информацией и SVG файлы могут свободно быть отредактированы с помощью обычного текстового редактора, существуют графические редакторы, специализированные под работу с векторной графикой и, очевидно, будут поддерживать данный формат и в некоторых сферах деятельности рекомендованы к использованию. Просмотр изображений в данном формате возможен с помощью любого современного браузера.

В настоящее время SVG имеет несколько версий для лучшей адаптации к различным аппаратным требованиям. Так, версии «SVG Tiny» и «SVG Basic» были созданы специально для мобильных устройств с ограниченными ресурсами. В то же время профиль «SVG Print» предназначен для сред печати документов.

Для анимации SVG-изображения W3C рекомендует стандарт «SMIL». Помимо официальной рекомендации, существуют и другие решения, такие как «ECMAScript». Распространенный формат для анимации векторной графики является Lottie. Lottie использует в основе JSON, в котором описываются шаги и какие в них атрибуты изменяются.

XML-элемент (но также и HTML-элемент) **<svg>** представляет собой контейнер для векторной графики SVG. На рисунке 2.1 можно увидеть информацию о широкой поддержки браузерами и на рисунке 2.2 таблицу с какой версии поддерживается элемент и его атрибуты.

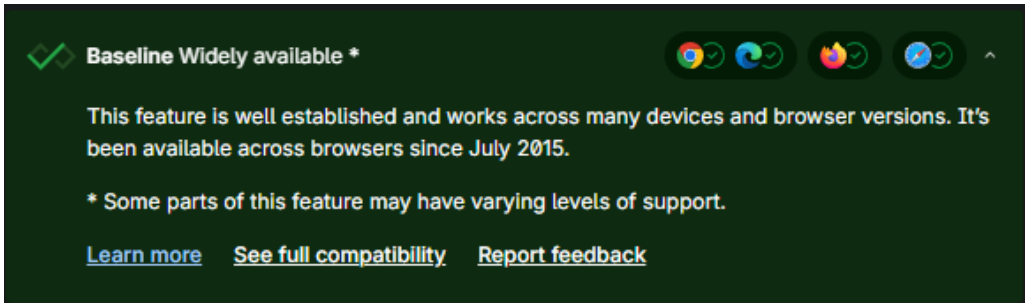


Рисунок 2.1: Информационное табло о широкой поддержки.

Browser compatibility

Report problems with this compatibility data • View data on GitHub

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet	WebView Android	WebView on iOS
svg	✓ 1	✓ 12	✓ 1.5	✓ 8	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 2	✓ 1	✓ 3	✓ 2
baseProfile	✓ 1	✓ 12	✓ 1.5	✓ 15	✓ 3	✓ 18	✓ 4	✓ 14	✓ 2	✓ 1	✓ 4.4	✓ 2
height	✓ 1	✓ 12	✓ 1.5	✓ 8	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 3	✓ 1	✓ 3	✓ 3
preserveAspectRatio	✓ 1	✓ 12	✓ 1.5	✓ 8	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 3	✓ 1	✓ 3	✓ 3
systemLanguage	✓ 1	✓ 12	✓ 12	✓ 12.1	✓ 3	✓ 18	✓ 14	✓ 12.1	✓ 2	✓ 1	✓ 3	✓ 2
version	✓ 1	✓ 12	✓ 1.5	✓ 15	✓ 3	✓ 18	✓ 4	✓ 14	✓ 2	✓ 1	✓ 4.4	✓ 2
viewBox	✓ 1	✓ 12	✓ 1.5	✓ 8	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 3	✓ 1	✓ 3	✓ 3
width	✓ 1	✓ 12	✓ 1.5	✓ 8	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 3	✓ 1	✓ 3	✓ 3
x	✓ 1	✓ 12	✓ 1.5	✓ 8	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 3	✓ 1	✓ 3	✓ 3
y	✓ 1	✓ 12	✓ 1.5	✓ 8	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 3	✓ 1	✓ 3	✓ 3
zoomAndPan	✓ 1	✓ 12	✓ 1.5	✓ 15	✓ 3	✓ 18	✓ 4	✓ 14	✓ 2	✓ 1	✓ 4.4	✓ 2

Рисунок 2.2: Таблица поддержки браузерами и с какой версии.

2.3. Примитивы в <svg>

Для самих примитивов есть соответствующие элементы, которые необходимо поместить внутрь элемента <svg>, чтобы они были нарисованы:

- **Прямоугольник** **<rect>**
- **Круг** **<circle>**
- **Эллипс** **<ellipse>**
- **Линия** **<line>**
- **Составная линия** **<polyline>**
- **Полигон** **<polygon>**
- **Путь** **<path>**

- **Линия**

```
<line x1=" " y1=" " x2=" " y2=" " stroke=" " stroke-width=" "></line>
```

Атрибуты элемента:

- **x1, y1, x2, y2** – координаты.
- **stroke** – цвет контура, в данном случае, самой линии.
- **stroke-width** – толщина контура, в данном случае, самой линии.
- **stroke-linecap** – стиль кончиков котура, в данном случае, самой линии. Доступны следующие стили: **butt** – конец линии прямой и заканчивается точно в указанной координате, данный стиль используется по умолчанию; **round** – конец линии закруглен и продлен на половину ширины линии; **square** – конец линии закруглен и продлен на половину ширины линии.
- **stroke-dasharray** – позволяет сделать линии пунктирной, через запятую указывается ширина пространство разделяющая линию.

Однако атрибуты отображения можно использовать в каскадных таблицах стиля (CSS) для более универсального применения и сокращения повторения кода:

```
<line style="stroke: ...; stroke-width: ..."></line>
```

```
<style>
.universal-style-class {
    stroke: ...;
    stroke-width: ...
}
</style>
...
<line class="universal-style-class"></line>
```

Ниже можно посмотреть примеры кодов и как будет выглядеть результат в таблице 2.1.

Таблица 2.1: Примеры использования элемента.

Код	Результат

`<svg>`
`<line stroke="red" stroke-width="1" x1="0" y1="10" x2="200" y2="10"></line>`
`<line stroke="black" stroke-width="2" x1="0" y1="20" x2="200" y2="20"></line>`
`<line stroke="blue" stroke-width="3" x1="0" y1="30" x2="200" y2="30"></line>`
`</svg>`	

`<svg stroke="#000" stroke-width="10">`
`<line stroke-linecap="butt" x1="10" y1="20" x2="200" y2="20"></line>`
`<line stroke-linecap="round" x1="10" y1="40" x2="200" y2="40"></line>`
`<line stroke-linecap="square" x1="10" y1="60" x2="200" y2="60"></line>`
`</svg>`	

`<svg stroke="#000" stroke-width="4">`
`<line stroke-dasharray="5,5" x1="10" y1="20"`
`x2="200" y2="20"></line>`
`<line stroke-dasharray="10,10" x1="10" y1="40"`
`x2="200" y2="40"></line>`
`<line stroke-dasharray="20,10,5,5,5,10" x1="10"`
`y1="60" x2="200" y2="60"></line>`
`</svg>`	

- **Составная линия**

```
<polyline points="x1 y1 x2 y2 ... xn yn"></polyline>
```

points – это координаты точек составной линии.

В таблице 2.2 можно увидеть пример использования элемента.

Таблица 2.2: Пример использования элемента.

Код	Результат

`<svg style="fill:none;stroke:#000;stroke-width:3">`
`<polyline points="20 20 40 25 60 40 80 120 120 140 200`
`180"></polyline>`
`</svg>`	

- **Круг, эллипс**

```
<circle cx=" " cy=" " r=" "></circle>

<ellipse cx=" " cy=" " rx=" " ry=" "></ellipse>
```

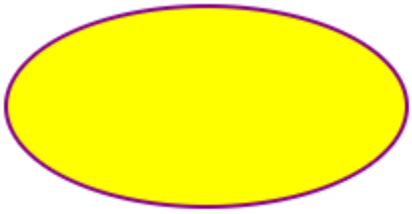
- **cx, cy** – координаты.
- **r** или **rx, ry** – радиус окружности.

В таблице 2.3 можно увидеть пример использование элементов

Таблица 2.3: Примеры использование элементов.

Код	Результат

`<svg>`
`<circle cx="50" cy="50" r="40" stroke="#080"`
`stroke-width="4" fill="#ff0"></circle>`
`</svg>`	

`<svg width="500">`
`<ellipse cx="200" cy="80" rx="100" ry="50" style="fill:#ff0;stroke:#808;stroke-width:2"></ellipse>`
`</svg>`
	

- **Прямоугольник**

```
<rect x=" " y=" " rx=" " ry=" " width=" " height=" "></rect>
```

  - **x, y** – координаты.
  - **rx, ry** – радиус, но используется для скругления углов.
  - **width, height** – ширина и высота

В таблице 2.4 можно увидеть пример использования элемента.

Таблица 2.4: Пример использования элемента.

Код	Результат

`<svg height="180">`
`<rect x="50" y="20" rx="20" ry="20" width="150" height="150" style="fill:red;stroke:black;stroke-width:5;opacity:0.5"></rect>`
`</svg>`	

- **Полигон**

```
<polygon points="x1 y1 x2 y2 ... xn yn"></polygon>
```

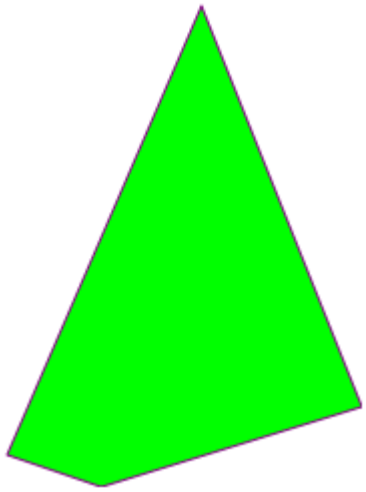
Такой же атрибут **points**, как и **<polyline>**. Дополнительно есть атрибут **fill-rule** – определяет каким образом будет выполняться заполнение фигуры. Доступные значения для атрибута – **evenodd** и **non-zero**.

Значительная разница между **<polyline>** – **<polygon>** замыкает фигуру последним ребром между последней точкой и первой.

В таблице 2.5 можно увидеть пример использования элемента.

Таблица 2.5: Пример использования элемента.

Код	Результат

`<svg height="250">`
`<polygon points="220 10 300 210 170 250 123 234" style="fill:#0f0;stroke:#808;stroke-width:1"></polygon>`
`</svg>`
	

- Путь

```
<path d="C x1 y1 x2 y2 c x3 y3 x4 y4 ... z"></path>
```

Универсальный элемент, позволяющий одним примитивов создать произвольную фигуру, в котором могут отверстия. Данный примитив можно часто встречать в различной векторной графике, используемой в веб-сайтах и/или веб-приложений или в целом в логотипах, иконках и т. д.

- **d** – набор команд и координат для построения фигуры. Значения перечисляются таким же образом, как и у атрибута points у <polyline>.
- **pathlength** – определяет общую длину пути.
- **transform** – преобразования над путем. Значения перечисляются таким же образом, как и у атрибутов d и points у <polyline>.

Команды атрибута d:

- **M** – move to.
- **L** – line to.
- **H** – horizontal line to.
- **V** – vertical line to.
- **C** – curve to.
- **S** – smooth curve to.
- **Q** – quadratic Bezier curve to.
- **T** – smooth quadratic Bezier curve to.
- **A** – elliptical Arc
- **Z** – close path.

Буквы команд могут быть написаны как с заглавной (большой), так и строчной (маленькой) буквы. Разница заключается в том, что в первом случае координаты будут

абсолютны и точкой отсчета будет нулевая, в другом случае координаты будут относительны последней точке предыдущей команды.

В таблице 2.6 можно увидеть пример использования элемента.

Таблица 2.6: Пример использования элемента.

Код	Результат

`<svg>`
`<path d="M 10 30 A 20 20 0 0 1 50 30 A 20 20 0 0 1 90 30 Q 90 60 50 90 Q 10 60 10 30 z"></path>`
`</svg>`	

`<svg>` `<path d="m 11.889623 3.5465535 c -4.8070431 0 -8.677416 4.2015656 -8.677416 9.4197585 v 49.302156 c 0 5.218194 3.8703729 9.419759 8.677416 9.419759 h 6.42673 V 26.359523 c 0 -5.218194 3.870374 -9.419759 8.677416 -9.419759 h 46.523549 v -3.973452 c 0 -5.2181929 -3.868894 -9.4197585 -8.675937 -9.4197585 z M 73.517318 16.939764 v 45.328704 c 0 5.218194 -3.868894 9.419759 -8.675937 9.419759 H 18.316353 v 3.97345 c 0 5.218194 3.870374 9.419759 8.677416 9.419759 h 52.950279 c 4.807043 0 8.677416 -4.201565 8.677416 -9.419759 V 26.359523 c 0 -5.218194 -3.870373 -9.419759 -8.677416 -9.419759 z"></path>` `</svg>`	

- **Текст**

```
<text x=" " y=" "> ... </text>
```

  - **x, y** – координаты, либо всего текста, либо можно указать для каждого символа чередованием, таким же образом как в атрибуте points у <polyline>.
  - **dx, dy** – отступ от предыдущего текстового элемента. Притом также, как и для всего текста, так и для каждого символа.
  - **rotate** – угол поворота.
  - **textLength** – максимальная ширина для коробки (элемента) для текста.
  - **lengthAdjust** – определяет как текст будет вмещаться в коробку (элемент).

2.4. Работа с SVG изображениями

Как уже ранее было выяснено, SVG формат – это текстовая информация использующее в основе синтаксис XML. Создать простое векторное изображение можно написав код в простом блокноте, после чего посмотреть результат в веб-браузере. Либо можно использовать другие инструмента для редактирования кода SVG изображения:

- **Веб-редактор P5.js** можно также использовать для написания код векторного изображения. Для этого необходимо нужно открыть список файлов «скетча», где находится кнопка можно увидеть на рисунке 2.3. После того, как появился список файлов «скетча» нужно переключиться на файл **index.html** и в нем очистить содержимое элемента **<body>** (в частности удалить элементы **<main>** и **<script>** со скриптом

«скетча» - **sketch.js**) и заместо предыдущего содержимого начать писать код SVG изображения. Удобство веб-редактора P5.js в том, что результат можно посмотреть буквально сразу, чем в связке с простым текстовым редактором и открытием в веб-браузере.

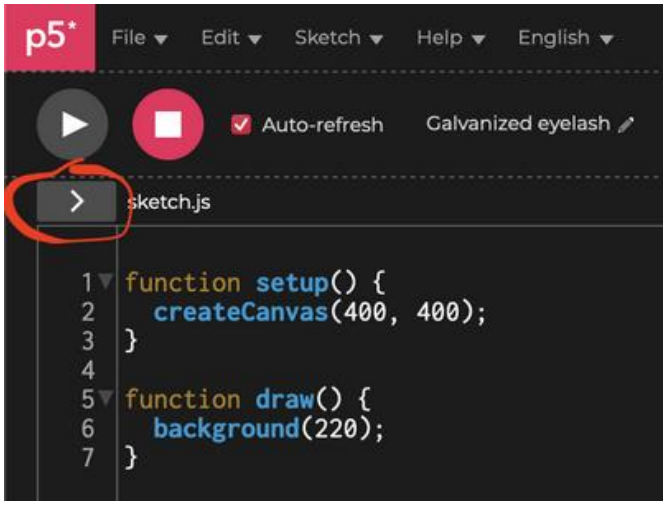


Рисунок 2.3: Кнопка для отображения файлов «скетча»

- **SVG Viewer** – простой редактор кода SVG изображения доступный в веб-браузере – https://www.svgviewer.dev/. Имеет удобные инструменты для импорта и экспорта изображений.

Более подходящий и рекомендуемый вариант работы с векторными изображениями – это использовать графический редактор, специализированный под работу с той самой векторной графикой. Такими редакторами являются следующие:

- **Inkscape** (бесплатный, открытый исходный код, рекомендуется) – программное обеспечение (ПО) доступное для операционных систем (ОС) Windows, Mac OS X или Linux – https://inkscape.org/. Обладает большим спектром функциональности для работы с векторной графики.
- **Adobe Illustrator** (требуется платная подписка Creative Cloud) – ПО доступное для ОС Windows и Mac OS X. Является широко используемым инструментом для работы с векторной графикой, чуть ли не в профессиональных сферах. Обладает широким спектром функциональности.

Практическая работа №2

Тема работы: Работа с векторной графикой

Основная задача: необходимо реализовать векторное изображение с композицией (2D сцена) и сохранить в формате SVG (.svg). Композиция должна состоять из, как минимум, одного объекта (или субъекта) и фона. Необходимо задействовать по одному из различных примитивов - <rect>, <circle>, <ellipse>, <line>, <polyline>, <polygon>, <path>; максимум 6 различных. Примитивы должны иметь различные атрибуты, работа должна быть подписана (фамилия, имя, группа) в правом нижнем углу экрана при помощи <text>. Варианты для реализации используйте из предыдущей работы.

Пример реализации:

```
<svg width="288" height="321" viewBox="0 0 288 321" fill="none" style="
filter:drop-shadow(20px 10px 5px rgba(0,0,0,.66))"
xmlns="http://www.w3.org/2000/svg">
<linearGradient id="grad1" x1="0%" y1="0%" x2="100%" y2="0%">
<stop offset="0%" style="stop-color:rgb(255,255,255);stop-opacity:1"></stop>
<stop offset="100%" style="stop-color:rgb(201,16,16);stop-opacity:5"></stop>
</linearGradient>
<path d="M235.806 221.399C199.733 279.997 132.807 303.604 86.4958 275.096C40.185
246.587 31.1209 176.2 67.1938 117.602C103.267 59.0032 170.193 35.3956 216.504
63.9043C262.815 92.413 271.879 162.8 235.806 221.399z" fill="url(#grad1)"
stroke="black" width="3"></path>
<ellipse cx="158" cy="250.5" rx="18" ry="15.5" fill="black"></ellipse>
<ellipse cx="97.5" cy="213.5" rx="13.5" ry="11.5" fill="black"></ellipse>
<circle cx="213" cy="181" r="18" fill="black"></circle>
<circle cx="213" cy="181" r="18" fill="black"></circle>
<circle cx="134.5" cy="133.5" r="22.5" fill="black"></circle>
<circle cx="134.5" cy="133.5" r="22.5" fill="black"></circle>
<line x1="167.328" y1="280.303" x2="188.328" y2="320.303" stroke="black" stroke-
width="3"></line>
<line x1="44.6949" y1="203.329" x2="0.694881" y2="226.329" stroke="black" stroke-
width="3"></line>
<line x1="50.6949" y1="150.329" x2="6.69488" y2="173.329" stroke="black" stroke-
width="3"></line>
<line x1="84.2833" y1="93.473" x2="6.28327" y2="108.473" stroke="black" stroke-
width="3"></line>
<line x1="254.222" y1="179.131" x2="286.222" y2="224.131" stroke="black" stroke-
width="3"></line>
<line x1="223.328" y1="234.303" x2="244.328" y2="274.303" stroke="black" stroke-
width="3"></line>
<ellipse cx="212.5" cy="62.5" rx="54.5" ry="41.5" fill="#1E1E1E"></ellipse>
<ellipse cx="191" cy="50.5" rx="10" ry="11.5" fill="white"></ellipse>
<ellipse cx="228" cy="73.5" rx="10" ry="11.5" fill="white"></ellipse>
<ellipse cx="191.5" cy="50.5" rx="3.5" ry="5.5" fill="black"></ellipse>
<ellipse cx="227.5" cy="73.5" rx="3.5" ry="5.5" fill="black"></ellipse>
<ellipse cx="235.5" cy="26" rx="17.5" ry="13" fill="#1E1E1E"></ellipse>
<line x1="227" y1="16" x2="227" stroke="black" stroke-width="2"></line>
```

```
<line x1="245.211" y1="25.3861" x2="259.211" y2="7.38606" stroke="black" stroke-
width="2"></line>
<line x1="91.5791" y1="274.73" x2="175.579" y2="143.73" stroke="#1E1E1E"></line>
</svg>
```

Результат выполнения кода можно увидеть на рисунке 2.4.

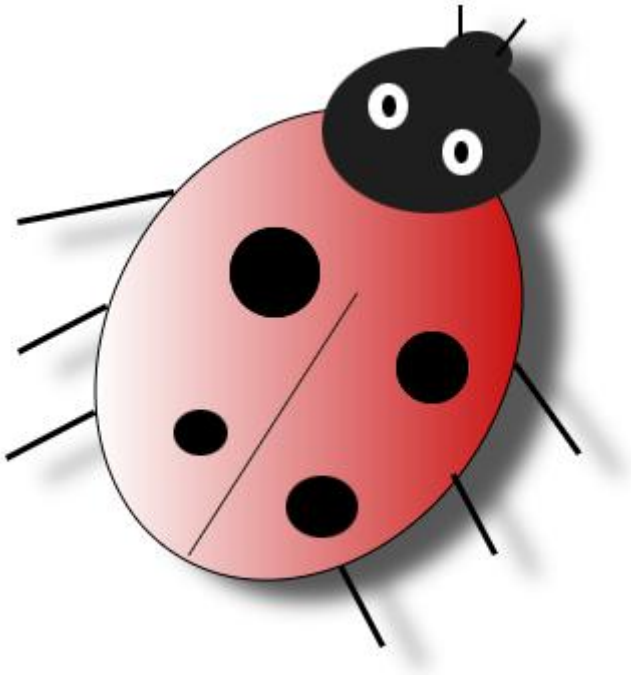


Рисунок 2.4: Результат выполнения кода.

Критерии оценивания:

- **Корректность кода и реализации (30%)** – оценка правильности использования графических примитивов SVG, таких как <прямоугольник>, <круг>, <эллипс>, <линия>, <полилиния>, <многоугольник> и <путь>. Проверка правильности, полноты и соответствия требованиям работы использования всех необходимых примитивов, а также правильности отрисовки и расположения графических элементов для формирования желаемой сцены.
- **Правильное использование атрибутов SVG (20%)** – оценка правильного применения атрибутов SVG, таких как заливка, обводка, толщина обводки, прозрачность, трансформация и другие, для настройки графических элементов. Проверка соответствия стилистических изменений каждой фигуре требованиям дизайна.
- **Креативность и сложность композиции (15%)** – оценка степени креативности и сложности созданного изображения, включая то, как примитивы комбинируются для создания интересной композиции. Оценивается степень оригинальности и сложности работы, т. е. то, как студент добавляет дополнительные детали для создания более интересной сцены.

- **Эстетика и организация сцены (10%)** – оценивается визуальное оформление сцены, включая расположение и композицию графических элементов. Визуальные детали соответствуют назначению и демонстрируют понимание принципов рисунка.
- **Оценка знаний (20%)** – объяснение процесса создания работы, которое может включать описание основных функций и используемой логики.
- **Соблюдение сроков сдачи работы (5%)** – будет учитываться соблюдение сроков сдачи работы, установленных преподавателем.

Вопросы для оценки знаний:

- Что такое SVG и чем векторная графика отличается от растровой?
- Перечислите графические примитивы, используемые в SVG, и объясните, как используется каждый из них.
- Как определяется прямоугольник в SVG и каковы основные необходимые атрибуты?
- Объясните роль атрибутов **fill**, **stroke** и **stroke-width**. Как они влияют на внешний вид графического примитива?
- Какое влияние оказывает атрибут **opacity** на элемент SVG?
- Как можно изменить размеры примитива SVG, такого как круг или прямоугольник, используя доступные атрибуты?
- Каковы преимущества и недостатки использования векторной графики по сравнению с растровой графикой?
- Как можно использовать функции **polyline** и **polygon** для создания сложных фигур в SVG?

III. Преобразования в 2D графике

Преобразования в графике — это набор математических операций, используемых для изменения положения, размера и ориентации фигур в двумерной плоскости. Эти операции имеют важное значение в компьютерной графике, поскольку позволяют манипулировать графическими объектами для создания анимации, применения визуальных эффектов и позиционирования элементов в сцене. К основным двумерным геометрическим преобразованиям относятся перемещение, вращение, масштабирование и отражение.

3.1. Смещение (перемещение)

Смещение (перемещение) — это процесс перемещения объекта без изменения его формы, размера или ориентации. Это достигается путем добавления постоянных значений к координатам каждой точки объекта.

В библиотеке P5.js есть функция **translate(x, y, *[z]*)**, которая позволяет реализовывать данный вид преобразования. Данная функция временно перемещает начальную точку координат для последующих функций рисования примитивов, тем самым к координатам самих объектов будет учитываться и коэффициент смещения заданным функцией translate().

3.2. Масштабирование

Масштабирование (увеличение или уменьшение) — геометрическое преобразование, изменяющее размер объекта путем умножения или деления координат каждой точки на коэффициент масштабирования. Оно может быть равномерным (когда коэффициенты масштабирования по осям X и Y равны) или неравномерным (когда коэффициенты масштабирования разные).

Для данного вида преобразования у P5.js есть функция **scale(s** или **x, *[y]*, *[z]*)**, которая, собственно, увеличивает или уменьшает все последующие объекты по коду «скетча».

3.3. Вращение (поворот)

Вращение — это поворот объекта вокруг фиксированной точки (обычно начала координат) на определённый угол.

Это изменяет ориентацию объекта, не изменяя его размер.

В P5.js данный вид преобразования выполняется через функцию **rotate(угол, *[ось]*)**, которая также применяется на все последующие объекты по коду «скетча».

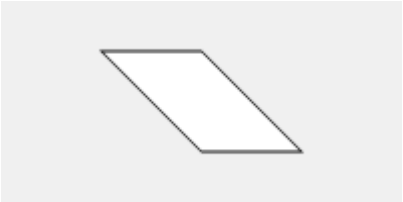
При помощи функции **transform()** можно задать фиксированную точку, вокруг которой будет вращаться объект. В качестве примера, есть официальный «скетч» от создателей P5.js, который демонстрирует на практике, доступен по этой ссылке – https://editor.p5js.org/p5/sketches/Transform:_Rotate.

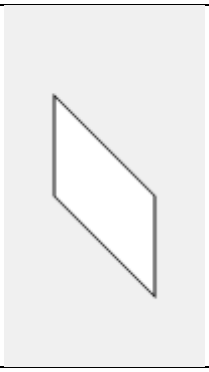
3.4. Искажение (деформация)

Функция сдвига или растяжения **shearX(угол)** и **shearY(угол)** деформируют фигуру вокруг осей X и Y, соответственно, на величину, заданную параметром **угла**. Объекты всегда деформируются относительно своего положения относительно начала координат, а положительные значения деформируют объекты по часовой стрелке. В таблице 3.1 можно увидеть примеры использования функций.

Таблица 3.1: Примеры использования функций shearX и shearY.

Код	Результат

`function setup() {`
`  createCanvas(200, 100);`
`  background(240);`
`  translate(width/4, height/4);`
`  shearX(PI/4);`
`  rect(0,0,50);`
`}`
	

`function setup() {`
`  createCanvas(100, 180);`
`  background(240);`
`  translate(width/4, height/4);`
`  shearY(PI/4);`
`  rect(0,0,50);`
`}`
	

3.5. push() и pop()

В p5.js функции **push()** и **pop()** используются для сохранения или восстановления настроек стиля и графических преобразований в сцене. Они позволяют изолировать преобразования и стили, применяемые к конкретным объектам, чтобы предотвратить их влияние на остальную часть сцены, и необходимы для создания сложных сцен без нежелательных преобразований во всей сцене.

- **push()** – сохраняет текущее состояние стиля (цвета, толщина линий и т. д.) и преобразований (перемещения, повороты, масштабирование) во временном «стеке». Эту функцию можно использовать в начале блока кода, где должны быть применены преобразования или стили, специфичные для объекта.
- **pop()** – восстанавливает состояние, ранее сохраненное функцией push(), удаляя любые изменения стиля или преобразования, примененные после push(). Она используется в конце блока кода для возврата к исходным настройкам, чтобы преобразования, примененные к объекту, не влияли на другие элементы в сцене.

Эти функции используются, когда в сложной сцене присутствует несколько объектов; push() и pop() помогают контролировать преобразования. Эти функции позволяют изменять только определенные объекты, не затрагивая при этом координаты, цвета или стиль всего эскиза.

3.6. Управлением временем

В p5.js существуют специальные механизмы, такие как: глобальная переменная **frameCount** и функция **millis()**, которые используются для измерения времени или для управления последовательностями анимации и событиями по времени в сценах. Хотя обе измеряют время, каждая имеет специфическое применение:

- **frameCount** – глобальная системная переменная, которая подсчитывает, сколько кадров (фреймов) было отрисовано с начала выполнения сцены (программы). Это указывает количество выполнений функции draw() с момента инициализации программы. frameCount начинается со значения 0 в setup() и увеличивается с каждым полным выполнением функции draw(); это полезно при создании анимации или эффектов по времени, поскольку каждое значение представляет собой определенный момент времени (с начала программы);
- **millis()** – функция, которая возвращает количество миллисекунд, прошедших с начала программы. Функция отслеживает, как долго выполняется эскиз в миллисекундах (тысячных долях секунды). Эта информация часто полезна для синхронизации событий и анимаций. Функция начинает отслеживать время до выполнения кода в функции setup(). Если ваш скетч включает функцию preload(), millis() начинает отслеживать время, как только начинается выполнение кода в preload(). Она используется для измерения реального времени и более точна для синхронизации или событий, не зависящих от кадра, поскольку на нее не влияют колебания frameCount (частоты кадров), что делает ее точным измерением прошедшего времени. Она используется для создания анимации, основанной на времени, или для запуска событий по истечении определенного промежутка времени.

3.7. applyMatrix()

Функция **applyMatrix()** использует вектор, как матрицу преобразования для применения преобразований над объектами. Данную функцию можно использовать для совмещения несколько видов преобразований в рамках одной функции.

Использование функции для реализации видов преобразования:

- **Смещение**

```
applyMatrix(1, 0, 0, 1, x, y);
```

- **Масштабирование**

```
applyMatrix(x, 0, 0, y, 0, 0);
```

- **Поворот**

```
var framerate = 60; // чем больше значение, там плавнее поворот
var angle = map(frameCount, 0, framerate, 0, TWO_PI);
applyMatrix(cos(angle), sin(angle), -sin(angle), cos(angle), 0, 0);
```

- **Совмещение смещения и масштабирования**

```
applyMatrix(scale_x, 0, 0, scale_y, move_x, move_y);
```

Практическая работа №3

Тема работа: Преобразования в 2D графике

Основная задача: необходимо реализовать динамическую композицию (2D сцену) при помощи, как минимум, по одному из трех видов преобразования – смещение, масштабирование и вращение. Можно взять Ваш вариант для реализации или использовать первую практическую работу.

Варианты для реализации:

1. **Анимированный маятник** – создайте маятник, который движется слева направо, имитируя естественное движение реального маятника, используя функцию **rotate()**. Маятник должен вращаться, как функция времени (**millis()** или **frameCount**).
   - **Цели:** Используйте вращение для анимации движения маятника.
   - **Подсказка:** Используйте **sin()** для медленного и естественного движения.
2. **Солнечная система** – создайте простую модель Солнечной системы, в которой планеты вращаются вокруг Солнца, используя **rotate()** и **translate()**. Каждая планета должна иметь свою собственную орбиту и собственное движение.
   - **Цели:** примените вращения и перемещения для имитации планетных орбит.
   - **Подсказка:** Планеты должны двигаться с разной скоростью.
3. **Воздушный змей с управлением мышью** – создайте анимацию, в которой воздушный змей движется по плоскости экрана, а его леска автоматически подстраивается под положение мыши. Используйте функции **translate()** и **rotate()** для правильного управления змеем в соответствии с движением мыши.
   - **Цели:** Используйте координаты **mouseX** и **mouseY** для управления воздушным змеем.
   - **Подсказка:** Добавьте плавное движение с помощью функции **lerp()** для хвоста.
4. **Анимация растущего цветка** – имитируйте рост цветка на экране. Начните с простого стебля, постепенно добавляя лепестки, которые растут и слегка поворачиваются. Используйте функции **scale()** и **rotate()** для анимации лепестков.
   - **Цели:** Анимируйте рост с помощью графических преобразований.
   - **Подсказка:** Управляйте скоростью роста с помощью **frameCount**.
5. **Интерактивная мозаика** – создайте мозаику из прямоугольников, которые меняют цвет и форму при наведении на них мыши. Используйте функции **translate()**, **rotate()** и **scale()** для изменения внешнего вида каждого прямоугольника при взаимодействии с мышью.
   - **Цели:** реализовать взаимодействие на основе положения мыши.
   - **Подсказка:** Каждый прямоугольник должен реагировать независимо на движение мыши.
6. **Оптическая иллюзия с вращениями** – создайте графический дизайн, создающий оптическую иллюзию с помощью кругов или линий, которые непрерывно вращаются и масштабируются. Анимация должна быть плавной и создавать впечатление кругового движения.
   - **Цели:** создать оптическую иллюзию посредством непрерывных вращений.

- **Подсказка:** Используйте **rotate()** и **scale()** для создания интересных эффектов.

7. **Фракталы с преобразованиями** – создайте фрактальную модель, в которой преобразования **rotate()**, **scale()** и **translate()** используются для генерации фрактального дерева или повторяющейся геометрической структуры. Структура должна генерироваться рекурсивно.

- **Цели:** использовать рекурсию для генерации фрактала.
- **Подсказка:** каждый уровень рекурсии должен уменьшать размер фигур.

8. **Каскад шаров** – смоделируйте каскад шаров, падающих на экран и отскакивающих от стены. Каждый шар должен вращаться во время падения и немного изменять свой размер при ударе о стену (эффект отскока).

- **Цели:** смоделируйте гравитацию и движение отскока с помощью вращения и масштабирования.
- **Подсказка:** используйте массив для одновременного управления несколькими шарами.

9. **Создание интерактивной анимации** – создайте интерактивную сцену, в которой пользователь может управлять преобразованиями объекта (масштабирование, вращение, перемещение) с помощью мыши и клавиатуры.

- **Цели:** используйте **mouseX**, **mouseY** для управления движением (перемещением), используйте клавиши для изменения размера (масштабирования) и угла поворота.
- **Подсказка:** объекты должны реагировать на действия пользователя в реальном времени.

10. **Создание динамической геометрической модели** – создайте сцену, в которой генерируются и анимируются несколько геометрических фигур (круги, прямоугольники, многоугольники) на основе набора преобразований. Каждый вновь добавленный объект должен масштабироваться и вращаться в соответствии со своим положением и динамически перемещаться по экрану.

- **Цели:** фигуры должны генерироваться случайным образом или на основе математического шаблона (например, спираль, сетка).
- **Подсказка:** используйте **random()** для генерации позиций или размеров и **frameCount** для анимации объектов на экране.

11. **Анимированные аналоговые часы** – создайте аналоговые часы, которые работают в реальном времени, используя преобразования **rotate()** и **translate()**. Стрелки часов должны быть анимированы и показывать точные часы, минуты и секунды.

- **Цели:** использовать функции **hour()**, **minute()**, **second()** из p5.js для определения положения стрелок.
- **Подсказка:** использовать функцию **rotate()** для анимации стрелок в зависимости от времени.

12. **Моделирование гравитации** – создать симуляцию, в которой несколько объектов падают на экран под действием гравитации, и когда они касаются дна, они подпрыгивают (эффект отскока).

- **Цели:** использовать графические преобразования для масштабирования объектов по мере их приближения ко дну (как если бы они были сжаты)

- **Подсказка:** объекты должны перемещаться вниз под действием гравитации. При ударе о землю их следует вращать или масштабировать, чтобы имитировать эффект отскока.

13. **Создание генеративного искусства** – создайте композицию генеративного искусства, использующую множество преобразований (перемещение, вращение, масштабирование) для генерации интересного геометрического узора на основе набора простых правил.

- **Цели:** использовать рекурсивные функции или циклы для генерации сложных структур.
- **Подсказка:** результат должен быть случайным при каждом запуске.

Пример реализации:

```
function setup() {
  createCanvas(600, 600);
  noStroke();
}
function draw() {
  background(200);
  // Объект 1: Прямоугольник, который вращается и масштабируется
  push();
  translate(width / 4, height / 2);
  rotate(radians(frameCount % 360));
  scale(sin(frameCount * .01) + 1.5); // Динамическое масштабирование
  fill(255, 0, 0);
  rect(-50, -50, 100);
  pop();
  // Объект 2: Круг, движущийся вдоль оси X и вращающийся
  push();
  translate((frameCount % width), height / 4);
  rotate(radians(frameCount % 360));
  fill(0, 255, 0);
  ellipse(0, 0, 80);
  pop();
  // Объект 3: Треугольник – движется по диагонали и масштабируется
  push();
  translate((frameCount % width) / 2, (frameCount % height) / 2);
  scale(abs(sin(frameCount * .01)) + .5); // Динамическое масштабирование
  fill(0, 0, 255);
  triangle(-30, 30, 30, 30, 0, -40);
  pop();
}
```

Критерии оценивания:

- **Реализация функций графических преобразований (30%)** – корректное использование функций rotate(), translate() и scale() в 2D-сцене для создания движения и динамических эффектов объектов. Реализация креативных преобразований, соответствующих контексту сцены.

- **Плавность и непрерывность анимации (10%)** – использование frameCount или millis() для управления движениями.
- **Сложность и разнообразие объектов в сцене (10%)** – визуальное взаимодействие между ними.
- **Оригинальность** в создании анимированной сцены и эффективное использование преобразований (10%).
- **Корректность кода (20%)** – проверка корректности кода, его оптимальности, отсутствия синтаксических и операционных ошибок.
- **Соблюдение сроков сдачи (10%)** – оценка по пунктуальности, если работа была сдана в установленный срок.
- **Оценка знаний (10%)** – пояснения относительно процесса выполнения работы, которые могут включать описание основных функций и используемой логики.

Вопросы для оценки знаний:

- Что такое геометрическое преобразование и почему оно важно в 2D-графике?
- Как функция translate() влияет на положение объекта в сцене?
- Какова роль функции rotate() в 2D-анимации и вокруг какой точки происходит вращение по умолчанию?
- Как работает функция scale() и как она влияет на размер и ориентацию объектов?
- Как можно использовать функции translate() и rotate() вместе для вращения объекта вокруг определенной точки в сцене?
- Как избежать влияния преобразований на все объекты в сцене?
- Как используются функции push() и pop() в p5.js и почему они необходимы при применении графических преобразований?
- Как можно одновременно применять несколько графических преобразований к объекту?
- В чем разница между frameCount и millis() и когда для анимации полезнее использовать одну из них вместо другой?
- Почему важно использовать функции push() и pop() при применении нескольких преобразований к объекту?
- Как управлять скоростью вращения объекта с помощью frameCount или millis()?
- Что произойдет, если применить несколько преобразований без использования push() и pop()? Как это повлияет на другие объекты на сцене?
- В чем разница между translate() и rotate() с точки зрения их влияния на положение и ориентацию объекта?
- Как создать зацикленную анимацию, используя функции sin() или cos() для масштабирования или вращения?
- Как использовать события мыши (например, mouseX и mouseY) для добавления интерактивности в сцену?

IV. 3D графика

4.1. Файлы *.OBJ

Формат файла OBJ — один из важнейших форматов файлов в приложениях 3D-печати и 3D-графики. Это предпочтительный формат для многоцветной 3D-печати и широко используется в качестве нейтрального формата обмена неанимированными 3D-моделями в графических приложениях.

Файл OBJ — это стандартный формат 3D-изображений, который можно экспортировать и открывать в различных 3D-редакторах. Он содержит трехмерные объекты, включая 3D-координаты, текстурные карты, полигональные грани и другую информацию об объектах.

Формат файла OBJ хранит информацию о 3D-моделях, может кодировать геометрию поверхности 3D-модели, а также хранить информацию о цвете и текстуре. Этот формат не хранит никакой информации о сцене, такой как положение источников света или анимация.

Файл OBJ генерируется программным обеспечением CAD (Computer Aided Design) как конечный продукт процесса 3D-моделирования. Расширение файла, соответствующее формату OBJ, — «*.OBJ».

Формат файлов OBJ является открытым и нейтральным, и часто используется для обмена 3D-моделями в графических приложениях, поскольку обеспечивает хорошую поддержку импорта и экспорта практически из всех программ САПР. Он также используется в качестве формата файлов для многоцветной 3D-печати, поскольку стандартный формат 3D-печати STL не включает информацию о цвете и текстуре.

Формат файлов OBJ был первоначально создан компанией Wavefront Technologies для своего приложения Advanced Visualizer для хранения геометрических объектов, состоящих из линий, многоугольников, кривых и поверхностей произвольной формы. Последняя документированная версия — v3.0, заменившая предыдущую версию v2.11.

Наиболее распространенным форматом в мире 3D-печати является STL. Однако STL — это старый формат файлов, который, несмотря на свою популярность, не учитывает современные требования. Точность 3D-печати приближается к разрешению на уровне микронов, и многоцветные модели становятся все более популярными. Формат файлов STL не может поддерживать высокое разрешение, поскольку более высокое разрешение увеличивает размер файла. Формат STL также не подходит для многоцветной 3D-печати, поскольку он не поддерживает информацию о цвете и текстуре. В отличие от него, формат OBJ позволяет достаточно точно аппроксимировать геометрию поверхности без существенного влияния на размер файла. Это возможно благодаря использованию кривых Безье и метода, называемого NURBS. Кроме того, формат OBJ имеет встроенную поддержку нескольких цветов и текстур в одной модели.

Таким образом, использование формата OBJ имеет ряд преимуществ по сравнению с использованием формата STL, когда требуются многоцветные модели

высокого разрешения. С другой стороны, формат файлов OBJ не так универсален, как формат STL. Практически все 3D-принтеры поддерживают формат STL. То же самое нельзя сказать о формате OBJ, хотя он и поддерживается. Поэтому, если вам нужно напечатать монохромную 3D-модель на стандартном принтере, предпочтительнее использовать формат STL.

4.2. Описание основных объектов и методов при работе с файлами *.OBJ в онлайн-редакторе p5.js

Библиотека p5.js — это библиотека JavaScript, которая упрощает разработку программ, использующих 2D и 3D графические элементы, и предназначена как для опытных, так и для начинающих разработчиков. Библиотека p5.js бесплатна и имеет открытый исходный код.

Библиотека p5.js обладает полным набором графических функций. С помощью этой библиотеки вся страница браузера отображается как рабочее пространство, в котором можно использовать объекты HTML5 для текста, видео, веб-камеры и звука.

Основные функции:

Загрузка 3D-модели из файла OBJ или STL

Функция loadModel(): должна быть помещена внутрь функции **preload()**, это позволяет полностью загрузить модель перед завершением программы.

Одним из ограничений форматов OBJ и STL является то, что они не учитывают масштаб. Это означает, что модели, экспортированные из разных программ, могут иметь разные размеры. Если модель не отображается, вы можете вызвать **loadModel()** с параметром normalization, установленным в true. Это изменит размер модели до подходящего масштаба для p5. Дополнительные изменения размера модели также можно внести с помощью функции **scale()**.

Аналогично, цветные STL-файлы не поддерживаются. Цветные STL-файлы будут отображаться без цветовых свойств.

Синтаксис:

```
loadModel(path, normalize, [successCallback], [failureCallback], [fileType]);
loadModel(path, [successCallback], [failureCallback], [fileType]);
```

где:

path String – путь к загружаемой модели;

normalize boolean – если true, масштабировать модель до стандартного размера при загрузке;

successCallback Function(p5.Geometry) – функция, которая будет вызвана после загрузки модели. Будет возвращен объект 3D-модели. (Необязательно);

failureCallback Function(Event) – вызывается с событием ошибки, если модель не загрузилась (необязательно);

fileType String – расширение файла модели (.stl, .obj) (необязательно). Возвращает p5.Geometry – объект типа p5.Geometry.

Функция model() – отображает 3D-модель на экране;

Синтаксис:

```
model(model);
```

где:

model – p5.Geometry.

Загруженная 3D-модель для рендеринга:

Файлы эскиза -> Загрузить файл

4.3. Создание статической 3D сцены

Задача практической работы состоит в проектировании и создании статической 3D-сцены с помощью онлайн-редактора p5.js, используя 3D-модели объектов, хранящиеся в файлах .OBJ, созданных индивидуально или загруженных из интернета.

Для иллюстрации хода практической работы поставлена цель создать статическую сцену, представляющую собой ветряную мельницу, окруженную забором, во дворе которой будут находиться несколько животных, дерево и стог сена. 3D-модели этих объектов можно создать или импортировать из репозитория 3D Warehouse, доступ к которому можно получить из главного меню 3D-графического редактора Google SketchUp ***Window->3D Warehouse***, а затем отредактировать по мере необходимости, как показано на рисунке 4.1.

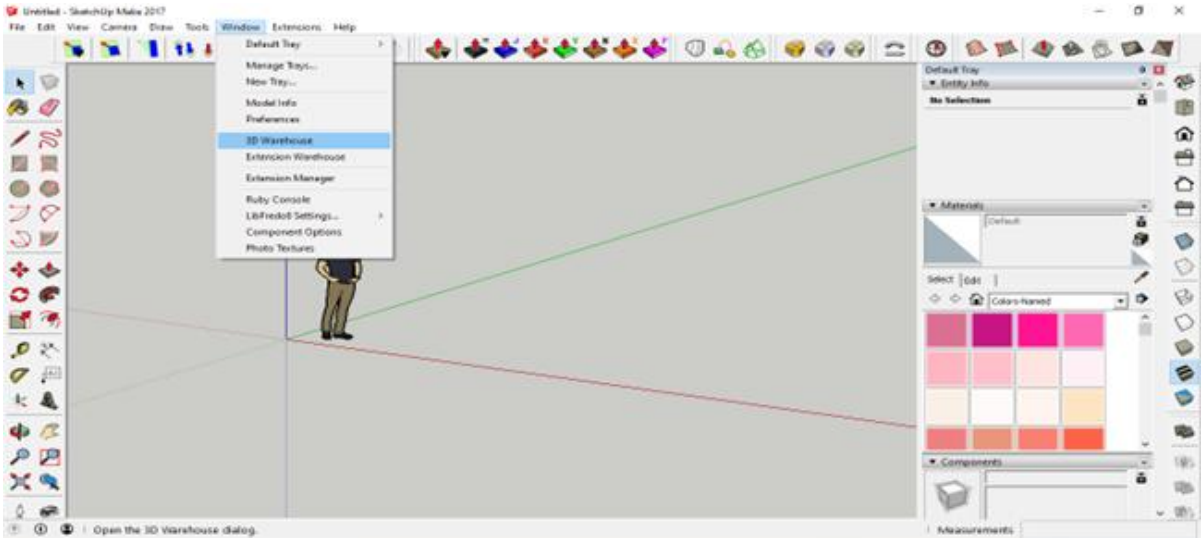


Рисунок 4.1: Рабочее окно репозитория в 3D Warehouse.

Для загрузки или импорта выбранной модели в текущий эскиз пользователь должен иметь учетную запись Google и пройти аутентификацию в среде 3D Warehouse. После загрузки необходимых моделей их можно редактировать по мере необходимости и экспортировать в файлы .OBJ с помощью плагина ***LIPID OBJ Exporter***, который можно установить из меню *Window->Extension Warehouse*. На рисунке 4.2 в поле поиска, в котором мы указываем ключевое слово OBJ, отображаются плагины, удовлетворяющие критериям поиска, среди которых можно увидеть искомый плагин *LIPID OBJ Exporter*, как показано на рисунке 4.3.

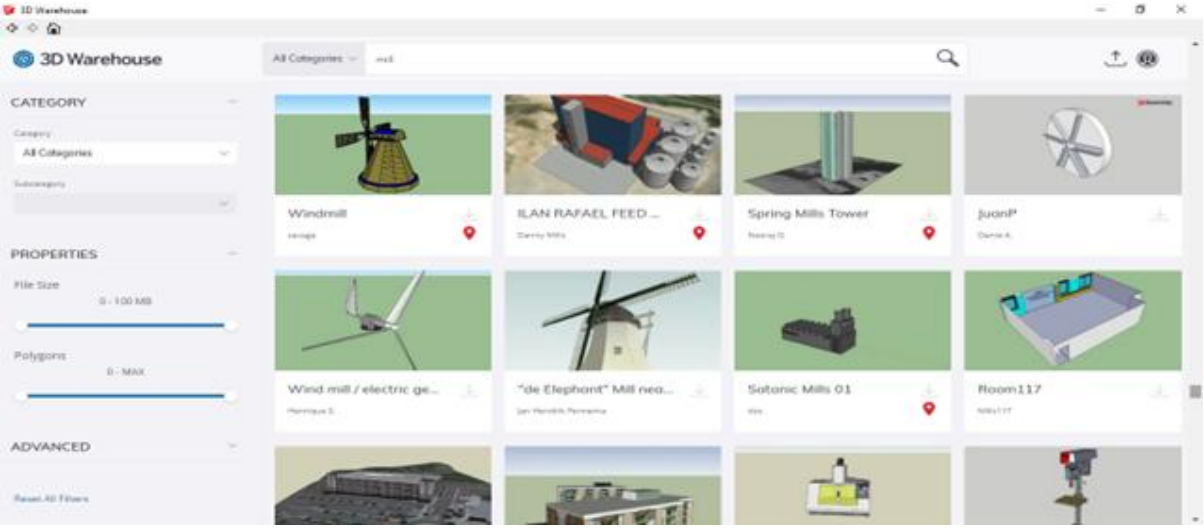


Рисунок 4.2: Поиск 3D моделей в 3D Warehouse.

Для установки плагинов пользователю необходимо иметь учетную запись Google и пройти аутентификацию в онлайн-репозитории *Extension Warehouse*.

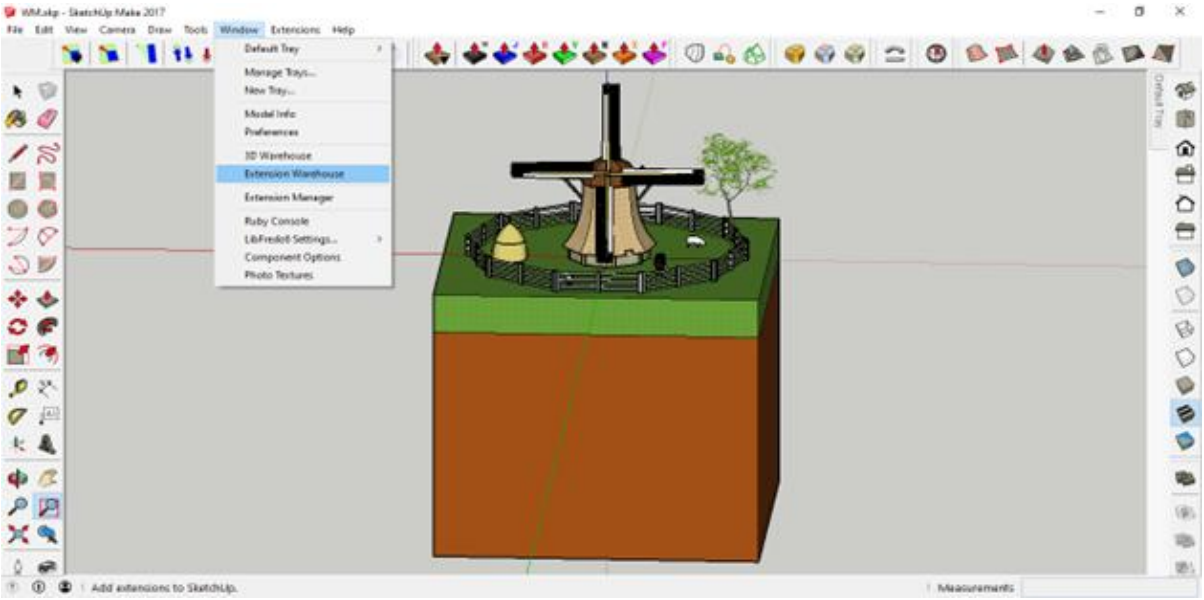


Рисунок 4.3: Пункт меню Extension Warehouse из полосы меню.

На рисунках ниже показаны шаги, которые необходимо выполнить для создания модели в формате *.obj.

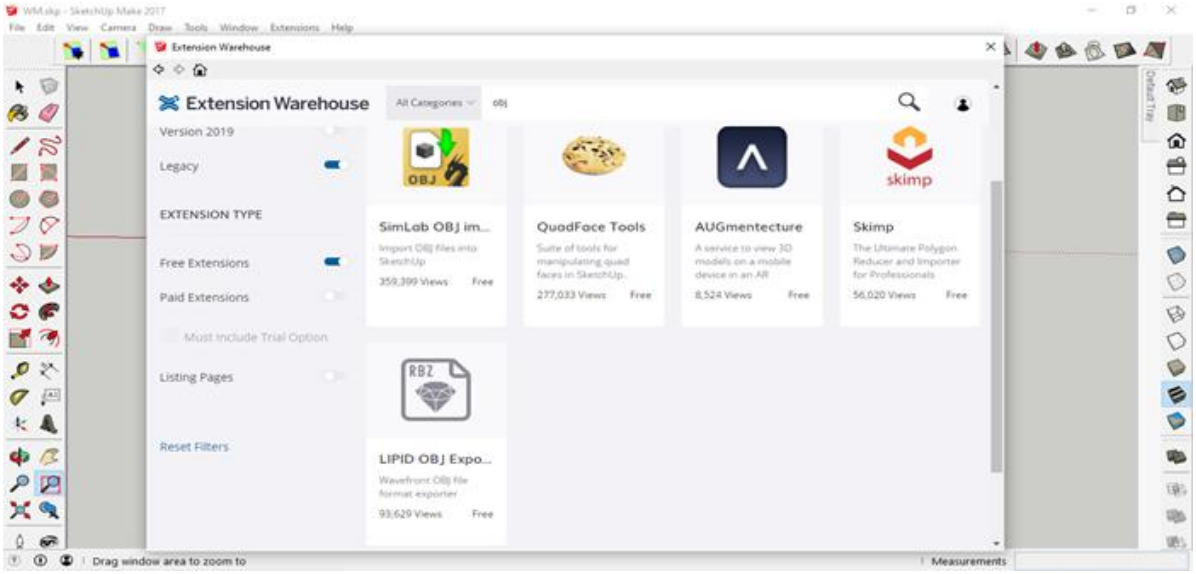


Рисунок 4.4: Окно поиска и установки плагинов из онлайн-репозитория Extension Warehouse.

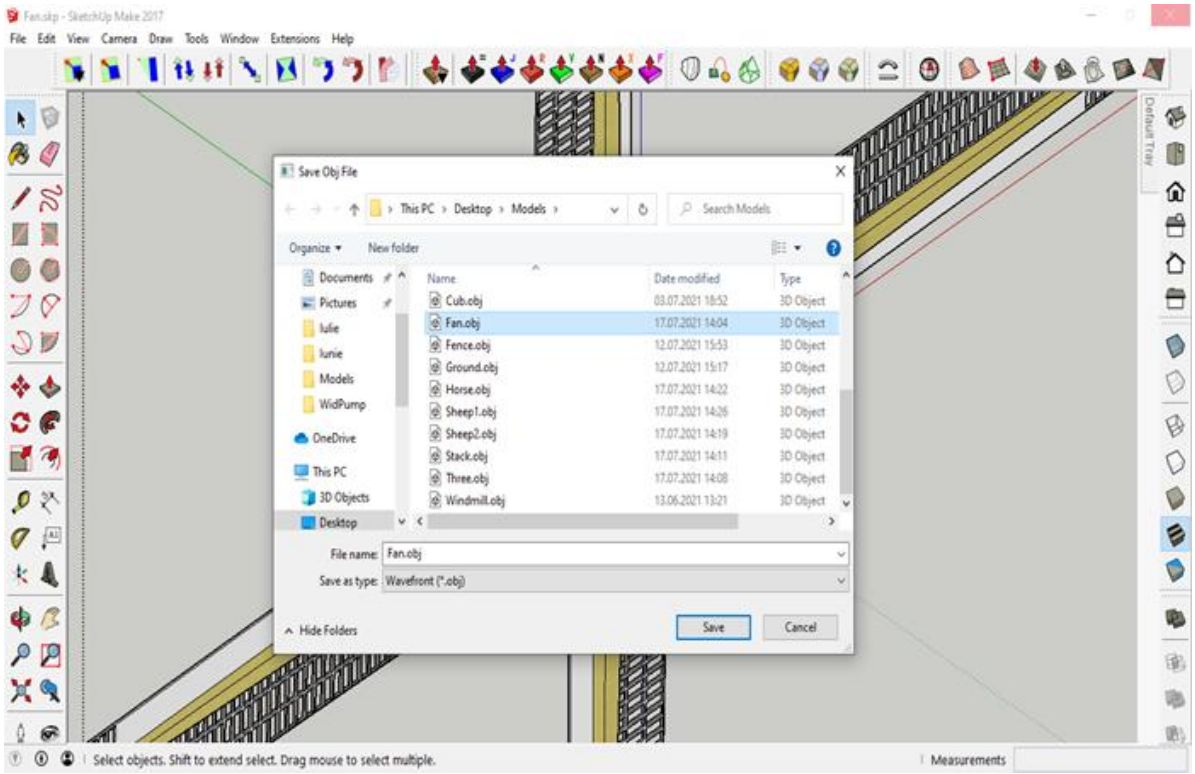


Рисунок 4.5: Окно для сохранения 3D-моделей объектов сцены в формате .obj.

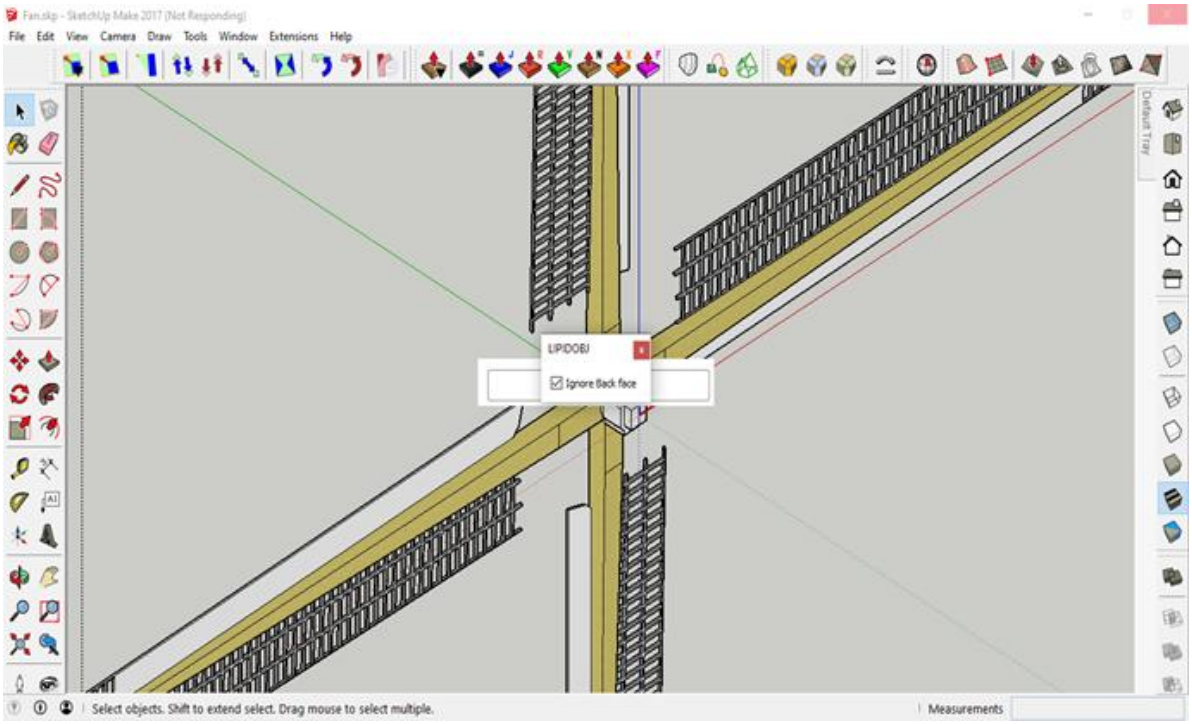


Рисунок 4.6: Опция игнорирования граней в плагине LIPIDOBJ при экспорте 3D-моделей в формат .obj.

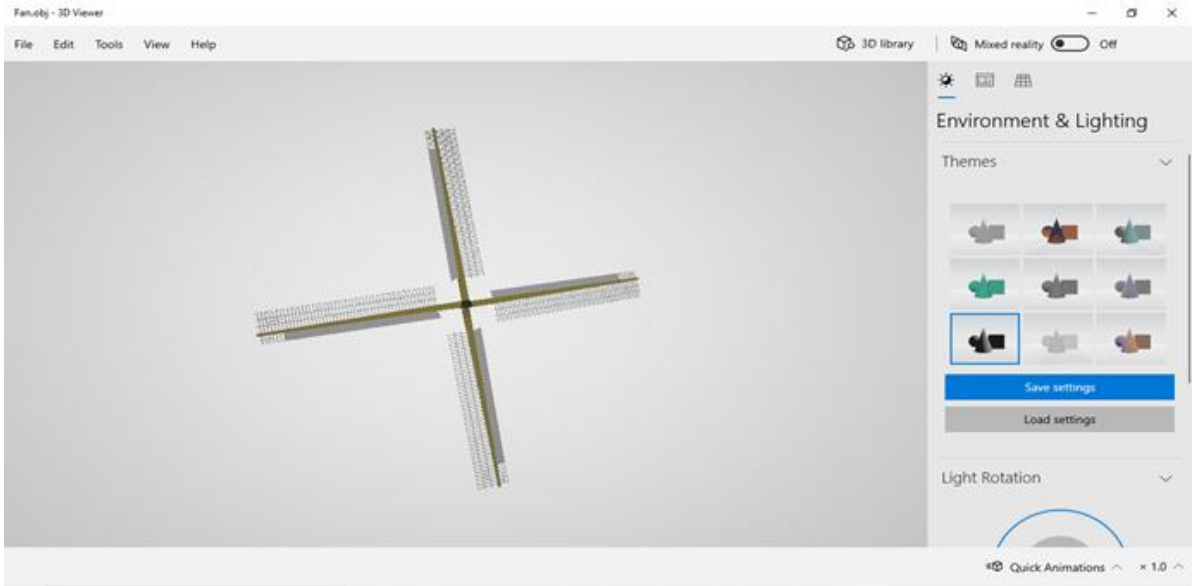


Рисунок 4.7: Просмотр содержимого файлов .OBJ с помощью стандартного приложения операционной системы Windows 10 — 3D Viewer.

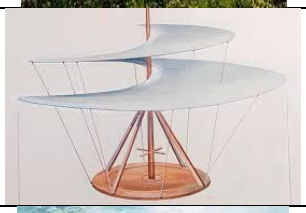
Практическая работа №4

Тема работы: Работа с 3D графикой

Основная задача: реализовать 3D композицию (сцену) из, как минимум, одного составного объекта и фона. Варианты для реализации предоставлены в таблице 4.1.

Таблица 4.1: Варианты для реализации.

Вариант	3D Объект	Модель
1	Карусель	
2	Маяк	
3	Бронемашина	
4	Водяная мельница	
5	Пропеллерный самолет	

Вариант	3D Объект	Модель
6	Флюгер	
7	Ветряная турбина	
8	Вертолет	
9	Подводная лодка	
10	Катапульта	

Пример реализации:

```
let fr = 30;
let obj;
function preload() {
  mill = loadModel('Windmill.obj');
  fan = loadModel('Fan.obj');
  gnd = loadModel('Ground.obj');
  sheep1 = loadModel('Sheep1.obj');
  sheep2 = loadModel('Sheep2.obj');
  horse = loadModel('Horse.obj');
  stack = loadModel('Stack.obj');
```

```
  fence = loadModel('Fence.obj');
  three = loadModel('Three.obj');
}
function setup() {
  createCanvas(400, 400, WEBGL);
  normalMaterial();
  frameRate(fr);
  angleMode(DEGREES);
}
function draw() {
  orbitControl();
  background(50);
  pointLight(255, 255, 255, 100000, 100000, 100000);
  noStroke();
  push();
  scale(0.005);
  translate(0, 0, 0);
  model(mill);
  pop();
  push();
  scale(0.005);
  translate(0, 3000, 13000);
  rotateX(15);
  model(fan);
  pop();
  push();
  scale(0.005);
  model(gnd);
  pop();
  push();
  scale(0.005);
  translate(-3000, 15000, 0);
  rotateZ(-10);
  model(sheep1);
  pop();
  push();
  scale(0.005);
  translate(10000, 0, 0);
  rotateZ(180);
  model(sheep2);
  pop();
  push();
  scale(0.005);
  rotateZ(-15);
  translate(7000, 12000, 0);
  model(horse);
  pop();
  push();
  scale(0.005);
  translate(-12000, -2000, 0);
  model(stack);
```

```
  pop();
  push();
  scale(0.005);
  translate(0, 0, 0);
  model(fence);
  pop();
  push();
  scale(0.005);
  translate(15000, -15000, 0);
  model(three);
  pop();
}
```

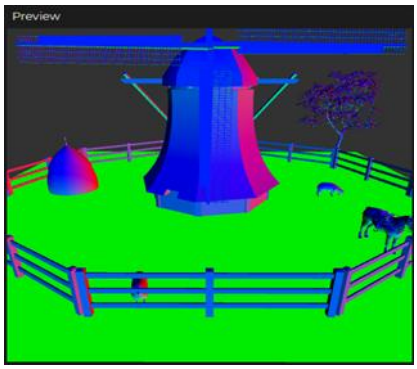


Рисунок 4.8: Результат выполнения кода.

Критерии оценивания:

- **Корректность 3D-моделирования в графическом редакторе (20%)** – соответствие созданного 3D-объекта требуемым параметрам и его корректность, а также корректность и отсутствие потери данных при экспорте в формат .obj.
- **Интеграция 3D-модели в p5.js (25%)** – функциональность импортированной модели в p5.js без ошибок отображения. Правильность и соответствие позиционирования модели в 3D-сцене.
- **Добавление и использование графических примитивов в p5.js (25%)** – добавление в сцену 3D-графических примитивов, способствующих общей композиции. Правильное позиционирование и размеры примитивов для создания целостной сцены.
- **Соответствие требованиям и ясность кода (10%)** – ясность, хорошая организация и документированность кода, соответствие указанным техническим требованиям. При необходимости для уточнения функциональности включены пояснительные комментарии.
- **Оценка знаний (10%)** – объяснения процесса создания работы, которые могут включать описание основных функций и логики.
- **Соблюдение требований и сроков сдачи итоговой презентации проекта** – (10%)

Вопросы для оценки знаний:

- Что такое статическая 3D-модель и чем она отличается от анимированной?
- Опишите шаги, необходимые для экспорта 3D-модели в формате .obj и её импорта в проект p5.js.
- Какие существуют 3D-графические примитивы в p5.js? Приведите несколько примеров и возможных применений.
- Как вращать, перемещать и масштабировать 3D-объект в p5.js?
- Каковы преимущества использования p5.js для рендеринга статических 3D-сцен?
- Как можно настроить освещение в 3D-сцене p5.js для достижения реалистичного эффекта?
- Какие функции и методы вы использовали для добавления 3D-графики в p5.js и как они повлияли на внешний вид финальной сцены?

V. Динамические 3D сцены

5.1. Введение в дополненную реальность

Дополненная реальность (AR) — это технология, которая объединяет виртуальные элементы с реальным миром, добавляя визуальные, звуковые или другие цифровые данные к тому, что пользователь видит в окружающей среде. Этот процесс обычно осуществляется с помощью таких устройств, как смартфоны, планшеты, очки дополненной реальности или другое оборудование, которое может проецировать или накладывать цифровые объекты поверх реальных изображений.

Дополненная реальность объединяет несколько технологий для создания иммерсивного приложения, которое включает в себя следующие аспекты:

1. Обнаружение и отслеживание — определение положения объектов в реальном мире для проецирования виртуальных элементов в точно заданное место.
2. Датчики и камеры — устройства используют камеры, акселерометры и гироскопы для сбора информации об окружающей среде.
3. Отображение — дополненные данные проецируются на экран устройства или через очки дополненной реальности, накладывая виртуальные объекты на физическую реальность.
4. Программное обеспечение и алгоритмы распознавания — приложения используют алгоритмы для анализа и обработки полученных данных, идентификации объектов и правильного позиционирования виртуальных элементов.

Таким образом, дополненная реальность — это технология, которая накладывает цифровые элементы, такие как 2D или 3D графические объекты, звуки и другие визуальные эффекты, на реальный мир. В данной статье будет рассмотрена платформа Artivive для создания дополненной сцены. Пример приложения дополненной реальности показан на рисунке 5.1.

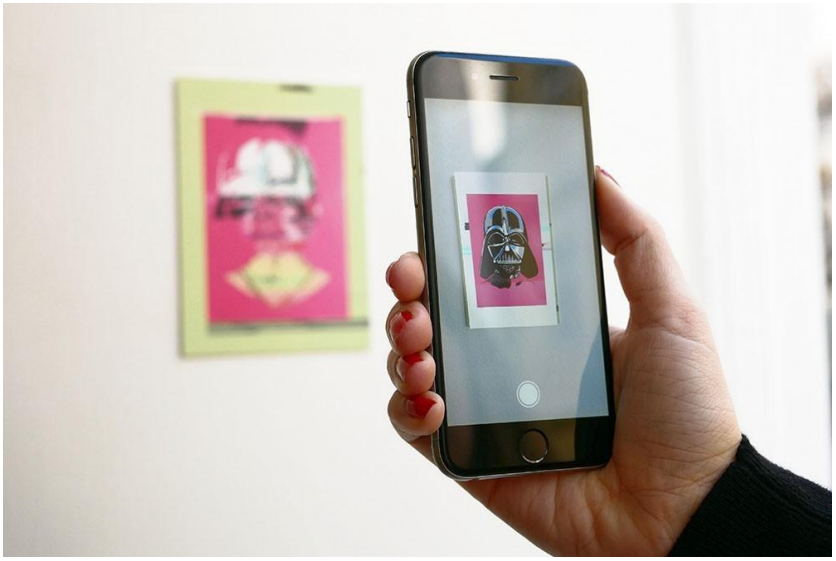


Рисунок 5.1: Дополненная реальность.

Для реализации проектов в дополненной реальности (AR) существует несколько популярных и универсальных платформ, каждая из которых предлагает функциональные возможности, адаптированные к различным типам проектов. Вот некоторые из известных платформ:

- **Unity + Vuforia — Unity** — очень популярный игровой движок, позволяющий создавать интерактивные AR-приложения. **Vuforia** — это SDK для AR, используемый с Unity для распознавания изображений, отслеживания объектов и интеграции 3D-графики. Он обеспечивает большую гибкость, подходит для сложных приложений и предлагает обширную поддержку анимации и 3D-графики.
- **ARKit (Apple)** — это фреймворк, разработанный Apple для создания AR-приложений на iOS. Он отлично интегрируется с устройствами Apple, очень эффективен для распознавания лиц, отслеживания движений и интеграции с оборудованием Apple.
- **ARCore (Google)** — это SDK, предоставляемый Google для создания AR-приложений на устройствах Android. Он предлагает функции обнаружения плоскостей, оценки яркости и отслеживания движений; он бесплатен и совместим со многими устройствами Android.
- **Spark AR Studio (Meta)** используется для создания AR-фильтров для Instagram и Facebook. Доступная для дизайнеров и создателей контента, она позволяет создавать пользовательские фильтры и предлагает интуитивно понятный интерфейс для быстрой реализации.
- **Lens Studio (Snapchat)** — это платформа Snapchat для создания AR-фильтров, известных как «линзы». Проста в использовании, с визуальными инструментами для отслеживания лица, тела и объектов, и идеально подходит для быстрого создания интерактивных впечатлений.
- **WebAR (8th Wall, AR.js) — 8th Wall и AR.js** — это платформы, позволяющие создавать AR-приложения непосредственно в браузере, без необходимости установки приложений. Пользователи могут получить доступ к AR через веб-ссылки, не устанавливая приложения; это эффективное решение для маркетинга и веб-проектов с использованием AR.
- **Artivive** — это AR-платформа, специально разработанная для искусства и мультимедиа, используемая для наложения анимации и AR-эффектов на статические изображения. Идеально подходит для художников и создателей мультимедийного контента, она позволяет просто и быстро внедрять AR-эффекты на физические материалы, такие как плакаты или иллюстрации.
- **Adobe Aero** — это инструмент от Adobe, позволяющий создавать AR-проекты без кода. Простой в использовании для дизайнеров и хорошо интегрированный с другими приложениями Adobe, он подходит для творческих и художественных проектов.

Большое разнообразие платформ дополненной реальности отражает гибкость и огромный потенциал этой области, каждый инструмент предлагает решения, адаптированные к различным типам проектов и уровням опыта.

5.2. Artivive

Чтобы начать работу с платформой Artivive, выполните следующие шаги:

1. **Загрузите приложение:** приложение Artivive доступно для iOS и Android.
2. **Создайте учетную запись:** войдите на платформу Artivive и создайте учетную запись пользователя. Учетная запись позволит вам загружать проекты и просматривать их через мобильное приложение. На рисунке 5.2 показано окно платформы Artivive.

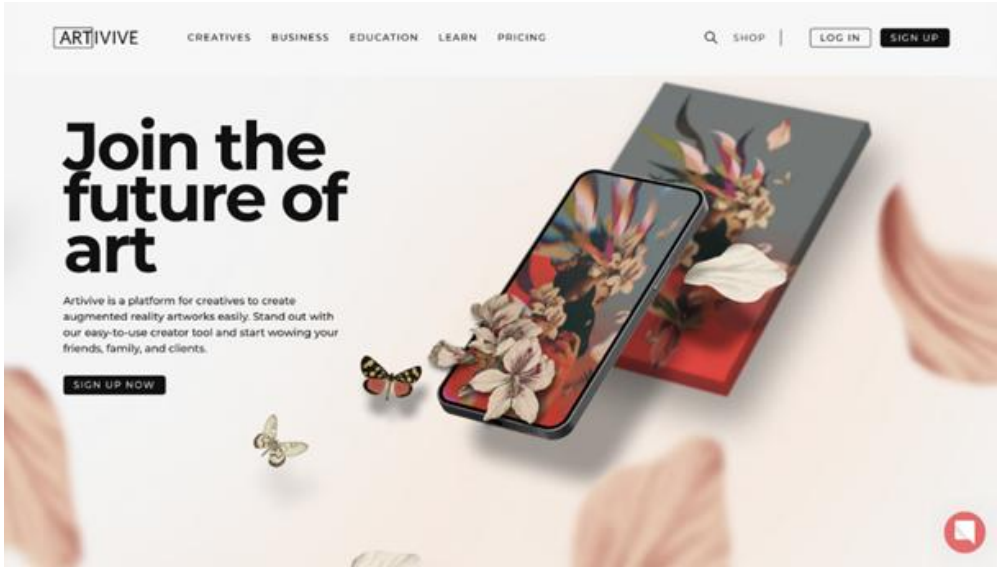


Рисунок 5.2: Платформа Artivive.

3. Чтобы узнать, как добавлять графику и накладывать ее на изображения, как показано на рисунке 5.3, перейдите в раздел **«Tutorials»** на платформе.

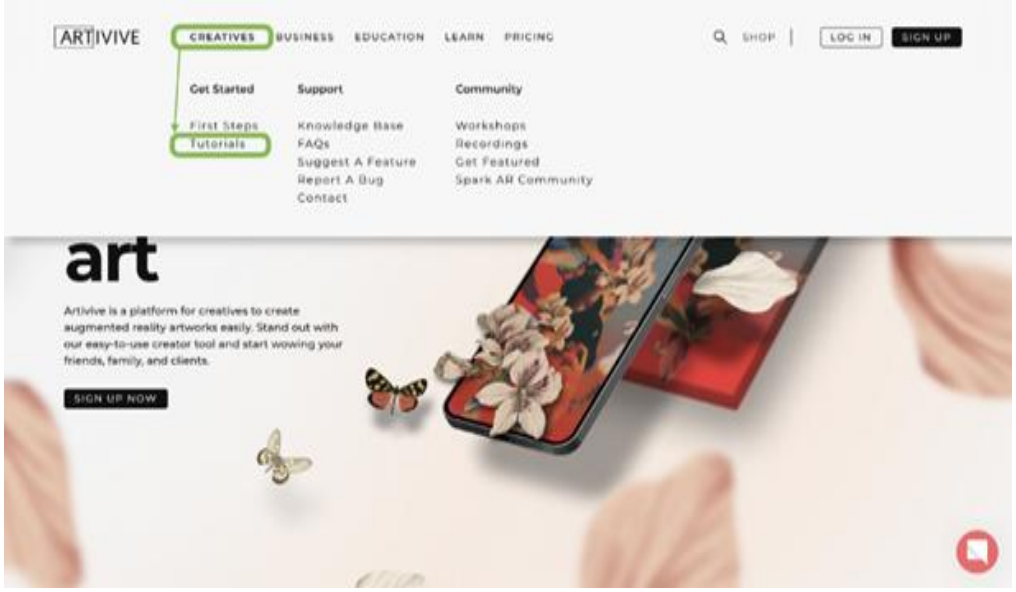


Рисунок 5.3: Раздел с обучающими материалами.

Следуйте инструкциям, доступным на платформе, чтобы создать первую простую дополненную сцену, которую можно отсканировать с помощью мобильного приложения,

чтобы понять, как она работает. Пример изображения до сканирования и изображения после сканирования показан на рисунке 5.4.

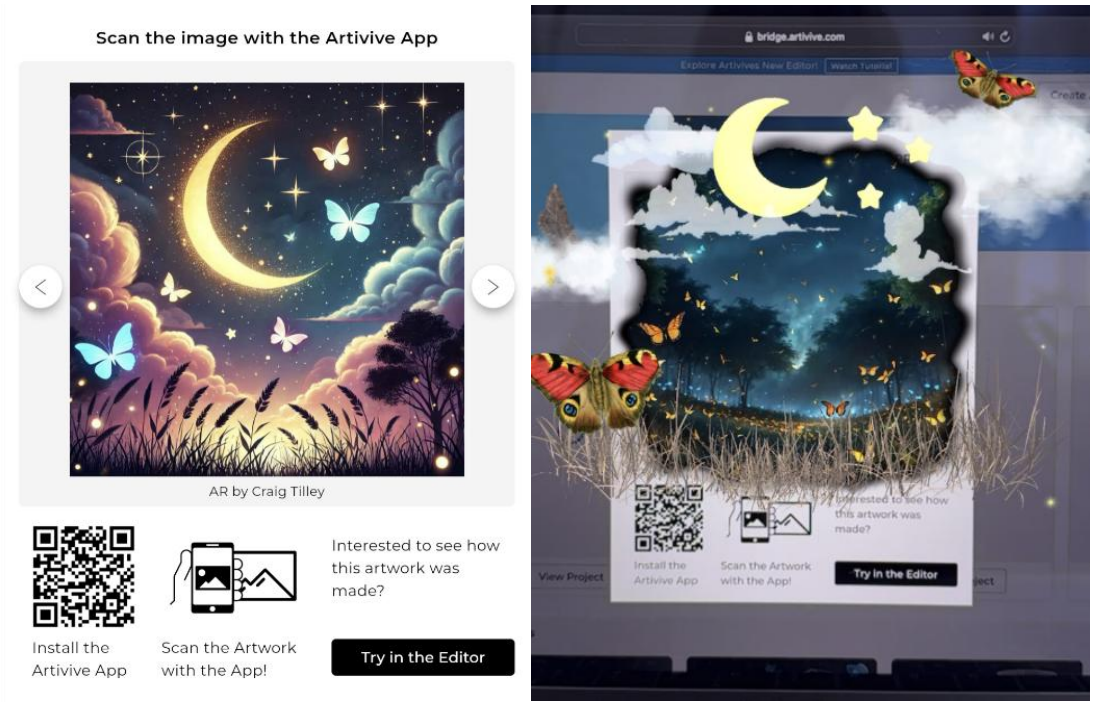


Рисунок 5.4: Дополненное изображение на стадии инициализации и сканирования.

Интерфейс платформы Artivive прост и интуитивно понятен, специально разработан для упрощения добавления элементов дополненной реальности (AR) на статические изображения.

Основные элементы интерфейса:

- **Панель загрузки** — раздел, где пользователи могут загружать базовое изображение (изображение-триггер) и контент AR (например, видео или анимацию).
- **Предварительный просмотр AR** — функция предварительного просмотра, позволяющая в режиме реального времени проверить, как выглядят эффекты AR, используя приложение Artivive на вашем телефоне.
- **Настройки проекта** — включают параметры для регулировки размера, продолжительности эффектов AR, а также другие параметры конфигурации.
- **Управление файлами** — пользователи могут управлять файлами и организовывать их для быстрого доступа и редактирования добавленного контента.

Пример интерфейса показан на рисунке 5.5.

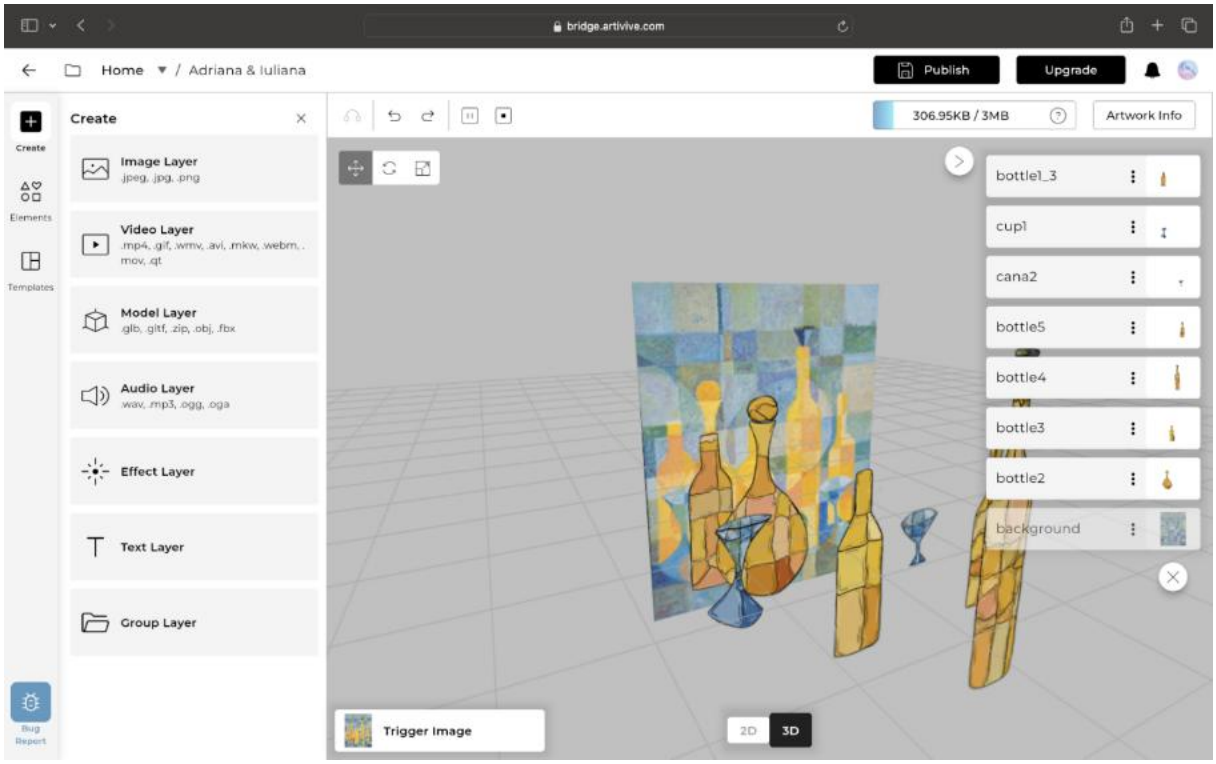


Рисунок 5.5: Интерфейс редактора Artivive.

Практическая работа №5

Тема работы: Реализация динамической 3D сцены.

Основная задача: создайте сцену дополненной реальности на платформе Artivive (или любой другой платформе для разработки AR-проектов), выбрав вдохновляющую вас тему. Сцена должна включать как 2D, так и 3D, статические и динамические графические элементы, а также звук или музыку для создания атмосферы. Сцена должна содержать не менее 5 графических элементов.

Руководство по выполнению работы:

- Определите тему: выберите для композиции тему, которая позволит проявить творческий подход.
- Добавьте графические объекты: используйте 2D и 3D элементы. Вы можете создать эти элементы самостоятельно или загрузить их из бесплатных онлайн-источников (например, Lumalabs.ai, Freepik, Pexels, Pixabay и т. д.).
- Интегрируйте эффекты и анимацию: добавьте движение и анимацию, чтобы сделать сцену динамичной. Платформа Artivive позволяет интегрировать видеоанимацию и интерактивные элементы.
- Добавьте звук: добавьте песню или звуковые эффекты, которые дополнят композицию. Убедитесь, что выбранные звуки не нарушают авторские права.
- Протестируйте и отрегулируйте: проверьте сцену с помощью приложения Artivive на своем телефоне, чтобы увидеть, как она выглядит, и внесите корректировки, если вас не устраивает результат.

Критерии оценивания:

- **Моделирование и анимация 3D-объектов (25%)** – соответствие проекта тематике и соблюдение минимального требования в 5 графических элементов, соответствие техническим требованиям, связанным с подготовкой файла для импорта в Artivive.
- **Интеграция динамических элементов и AR-эффектов (25%)** – креативное использование платформы Artivive, включая интеграцию графических элементов и анимации.
- **Эстетика и целостность динамической сцены (25%)** – эстетическая целостность композиции, благодаря гармоничному сочетанию визуальных элементов и звука, анимация и объекты четкие и качественно выполненные, с вниманием к деталям.
- **Умение тестировать и корректировать сцену** для достижения плавного и захватывающего пользовательского опыта. (20%)
- **Соблюдение сроков сдачи (5%)**

Вопросы для оценки знаний:

- Что такое дополненная реальность и чем она отличается от виртуальной реальности?
- Какова роль платформы Artivive в создании проектов дополненной реальности?
- Как добавить анимацию в проект дополненной реальности с помощью Artivive?
- В чем основные различия между 2D и 3D графическими объектами в контексте дополненной реальности?
- Как звук может влиять на пользовательский опыт в проекте дополненной реальности?
- Почему важно тестировать и корректировать сцену дополненной реальности на мобильном устройстве?
- Как можно использовать альтернативные платформы вместо Artivive для создания приложений дополненной реальности?

VI. Реализация динамических процессов в графике

Моделирование динамических процессов, помимо чисто научных целей, может иметь и прикладное значение.

Для физического моделирования динамических процессов можно использовать графическую библиотеку p5.js. Она позволяет моделировать механические системы (в рамках законов теоретической механики). С помощью p5.js можно моделировать поступательные и вращательные движения в трех плоскостях.

В представленном примере моделируется столкновение двух сфер массами m_1 и m_2, которые до столкновения имеют линейные скорости перемещения центров, обозначенные v_1 и v_2. После столкновения сферы будут двигаться со скоростями v'_1 и v'_2. Сферы катятся по твердой поверхности, т. е. совершают вращательное движение, заданное углами: «omega$_1$» и «omega$_2$». Эксперимент показан на рисунке 6.1.

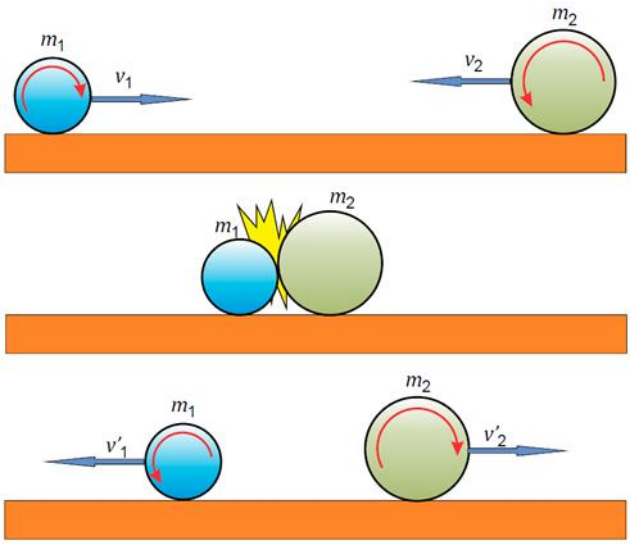


Рисунок 6.1: Столкновение двух сфер.

Используя закон сохранения механической энергии, представленный в формуле (6.1):

$$\frac{m_1 v_1^2}{2} + \frac{m_2 v_2^2}{2} = \frac{m_1 v'^2_1}{2} + \frac{m_2 v'^2_2}{2} \tag{6.1}$$

и формула закона сохранения импульса (6.2):

$$m_1 v_1 + m_2 v_2 = m_1 v'_1 + m_2 v'_2 \tag{6.2}$$

Соотношение скоростей до и после столкновения сфер можно определить с помощью формул 6.3 и 6.4:

$$v'_1 = -v_1 + 2\frac{m_1 v_1 + m_2 v_2}{m_1 + m_2} \tag{6.3}$$

$$v'_2 = -v_2 + 2\frac{m_1 v_1 + m_2 v_2}{m_1 + m_2} \tag{6.4}$$

Угол поворота можно выразить с помощью формул 6.5 и 6.6.

$$\text{omega}_1 = \frac{r_1}{v_1}; \tag{6.5}$$

$$\text{omega}_2 = \frac{r_2}{v_2}, \tag{6.6}$$

Модель данного эксперимента может быть описана следующим кодом:

```
let x1=-150;
let v1=5;
let m1=50;
let omega1=v1/m1;
let a1=0;
let x2=150;
let v2=-15;
let m2=20;
let omega2=v2/m2;
let a2=0;
function setup() {
  createCanvas(500, 500, WEBGL);
  strokeWeight(.2);
  frameRate(24);
}
function draw() {
  background(200);
  orbitControl();
  pointLight(255, 255, 255, 0, 0, 1000);
  push();
  box(500,100,100);
  pop();
  push();
  translate(x1, -100/2-m1, 0);
  rotateZ(a1);
  sphere(m1);
  pop();
  push();
  translate(x2, -100/2-m2, 0);
  rotateZ(a2);
  sphere(m2);
  pop();
  if(x1+m1>x2-m2) {
    v1=2*(m1*v1+m2*v2)/(m1+m1)-v1;
    v2=2*(m2*v2+m1*v1)/(m1+m1)-v2;
    x1=x1+v1;
    x2=x2+v2;
    omega1=v1/m1;
```

```
    omega2=v2/m2;
  }
  x1=x1+v1;
  x2=x2+v2;
  a1=a1+omega1;
  a2=a2+omega2;
}
```

В приведенной выше программе в качестве переменных мы имеем:

- **x1** и **x2** — начальные положения шаров.
- **v1** и **v2** — начальные линейные скорости центров шаров.
- массы шаров **m1** и **m2**, которые также выражают радиусы шаров.
- **omega1=v1/m1** и **omega2=v2/m2** — углы поворота шаров.
- **a1 = 0** и **a2 = 0** — начальные углы поворота шаров.

Тело, по которому катятся шары, создается с помощью функции **box(500,100,100)** — это статический объект.

Шары рисуются с помощью функции **sphere()**, к ним применяется поступательное движение с помощью функции **translate()**, а вращательное движение — с помощью функции **rotateZ()**.

Модели тел вызываются каждая в отдельном блоке, ограниченном вызовами **push()** и **pop()**.

Последняя часть программы включает формулы для расчета скоростей сфер, их положений и угла поворота.

6.1. Создание простых 3D графических примитивов

Для создания объектов в p5.js используются простые 3D-графические примитивы, такие как: **plane()**, **box()**, **sphere()**, **cylinder()**, **cone()**, **ellipsoid()** и **torus()**.

- **plane() –** рисует плоскость с заданной шириной и высотой.

  Синтаксис:

```
plane([width],[height], [detailX], [detailY]);
```

  где:

  - [width] – ширина (необязательно)
  - [height] – высота (необязательно))
  - detailX – количество треугольников по оси x (необязательно)
  - detailY – количество треугольников по оси y (необязательно)

- **box() –** рисует параллелепипед с заданной шириной, высотой и глубиной.

Синтаксис:

```
box([width],[Height],[depth],[detailX], [detailY]);
```

где:

- [width] – ширина фигуры (необязательно)
- [height] – высота фигуры (необязательно)
- [depth] – глубина фигуры (необязательно)
- detailX – количество треугольников по оси x (необязательно)
- detailY – количество треугольников по оси y (необязательно)

- **sphere() –** рисует сферу с заданным радиусом. DetailX и detailY определяют количество сегментов по осям x и y сферы. Большее количество сегментов делает сферу более гладкой. Максимально рекомендуемое значение — 24. Использование значения больше 24 может замедлить работу браузера.

Синтаксис:

```
sphere([радиус], [detailX], [detailY]);
```

где:

- радиус – радиус окружности (необязательно)
- detailX – количество сегментов по оси x (необязательно)
- detailY – количество сегментов по оси y (необязательно)

- **cylinder() –** рисует цилиндр с заданным радиусом и высотой. detailX и detailY определяют количество сегментов по осям x и y цилиндра. Большее количество сегментов делает цилиндр более гладким. Максимально рекомендуемое значение для detailX — 24. Использование значения больше 24 может замедлить работу браузера.

Синтаксис:

```
cylinder([радиус], [высота], [detailX], [detailY], [bottomCap], [topCap]);
```

где:

- радиус – радиус основания цилиндра (необязательно)
- высота – высота цилиндра (необязательно)
- detailX – количество сегментов по оси x, по умолчанию 24 (необязательно)
- detailY – количество сегментов по оси y, по умолчанию 1 (необязательно)
- bottomCap : Boolean – рисовать ли нижнюю часть цилиндра (необязательно)
- topCap : Boolean – рисовать ли верхнюю часть цилиндра (необязательно)

- **cone() –** рисует конус с заданным радиусом и высотой. detailX и detailY определяют количество сегментов по осям x и y конуса. Большее количество сегментов делает конус более гладким. Максимально рекомендуемое значение для detailX — 24. Использование значения больше 24 может замедлить работу браузера.

Синтаксис:

```
cone([radius], [height], [detailX], [detailY], [cap]);
```

где:

- радиус – радиус основания конуса (необязательно)
- высота – высота конуса (необязательно)
- detailX – количество сегментов по оси x, по умолчанию 24 (необязательно)
- detailY – количество сегментов по оси y, по умолчанию 1 (необязательно)
- cap : Boolean – следует ли рисовать основание конуса (необязательно)

- **ellipsoid()** – рисует эллипсоид. detailX и detailY определяют количество сегментов по осям x и y фигуры. Большее количество сегментов делает эллипсоид более плавным. Избегайте значений детализации больше 150, так как это может привести к сбою браузера.

Синтаксис:

```
ellipsoid([radiusx], [radiusy], [radiusz], [detailX], [detailY]);
```

где:

- radiusx – радиус эллипсоида по оси x (необязательно)
- radiusy – радиус эллипсоида по оси y (необязательно)
- radiusz – радиус эллипсоида по оси z (необязательно)
- detailX – количество сегментов по оси x, по умолчанию 24. Избегайте количества сегментов более 150, так как это может привести к сбою браузера. (Необязательно)
- detailY – количество сегментов по оси y, по умолчанию 16. Более 150 может привести к сбою браузера. (Необязательно)

- **torus() –** рисует кольцо. detailX и detailY определяют количество сегментов по осям x и y тора. Максимальные значения detailX и detailY по умолчанию равны 24 и 16 соответственно. Установка относительно небольших значений, таких как 4 и 6, позволяет создавать новые фигуры, помимо тора.

Синтаксис:

```
torus([radius], [tubeRadius], [detailX], [detailY]);
```

Параметры:

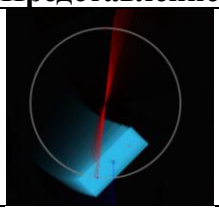
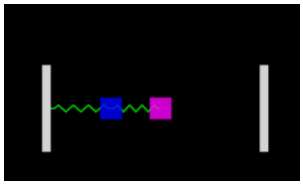
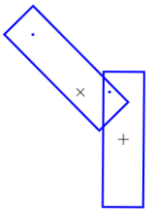
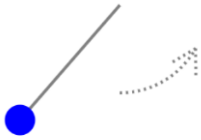
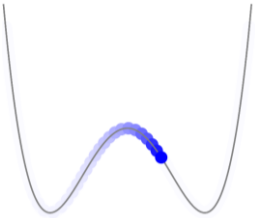
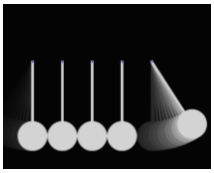
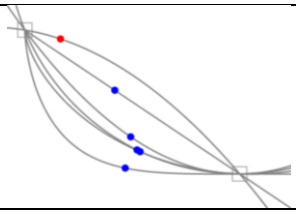
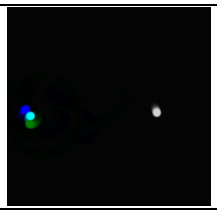
- radius – внешний радиус фигуры (необязательно)
- tubeRadius – внутренний радиус фигуры (необязательно)
- detailX – количество сегментов по оси x, по умолчанию 24 (необязательно).
- detailY – количество сегментов по оси y, по умолчанию 16 (необязательно).

Практическая работа №6

Тема работа: Реализация динамических процессов в графике.

Основная задача: Разработайте программу, которая создает сцену, в которой происходит физический процесс с использованием стандартных функций перемещения, масштабирования и/или вращения, в соответствии с вариантом, указанным в таблице 6.1.

Таблица 6.1: Варианты для реализации

№	Модель	Представление	№	Модель	Представление
1	Пружина одинарного растяжения		6	Жесткий маятник с круговым движением	
2	Двойная пружина растяжения		7	Жесткий двойной маятник	
3	Хаотический маятник		8	Столкновение жестких блоков	
4	Движение сферы по траектории, напоминающей горб.		9	Маятник Ньютона	
5	Движение сферы по нескольким траекториям		10	Привлекательность тел	

Критерии оценивания:

- **Корректность кода (20%)** — проверка правильности написания кода, отсутствия синтаксических и операционных ошибок.
- **Использование 3D-графических примитивов (20%)** — оценка правильности использования и вариативности графических примитивов в соответствии с требованиями работы.
- **Соответствие инструкциям и требованиям (10%)** — проверка корректности требований задачи, таких как определенные формы или размеры графических примитивов и их динамика.
- **Оптимизация кода (10%)** — оценка эффективности кода в использовании ресурсов и предотвращение избыточного кода.
- **Интерактивность (10%)** — например, реакции на действия мыши или клавиатуры; оценивается правильность реализации этих функций.
- **Визуальная эстетика (10%)** — анализ визуальной привлекательности композиции, такой как цветовая гармония, пропорции и графический баланс.
- **Соблюдение сроков (10%)** — оценка по пунктуальности, если работа была сдана вовремя.
- **Оценка знаний (10%)** - пояснения, предоставленные относительно процесса выполнения работы, которые могут включать описание основных функций и используемой логики.

Вопросы для оценки знаний:

- Перечислите простые 3D-графические примитивы.
- Перечислите функции, используемые для графических преобразований.
- Как можно изменить атрибуты отображения 3D-графических примитивов?
- Назовите форматы вращения и соответствующие функции.
- Какие графические примитивы можно использовать для создания базовых фигур в 2D-графической сцене?
- Как можно добиться комбинирования графических преобразований?
- Какова роль функций push() и pop() в обработке геометрических преобразований?
- Как включить 3D-режим в p5.js?